ALU Design Example (1-bit & 4-bit)

Md. Momenul Haque
Lecturer, CSE, BAUET

ALU in CPU

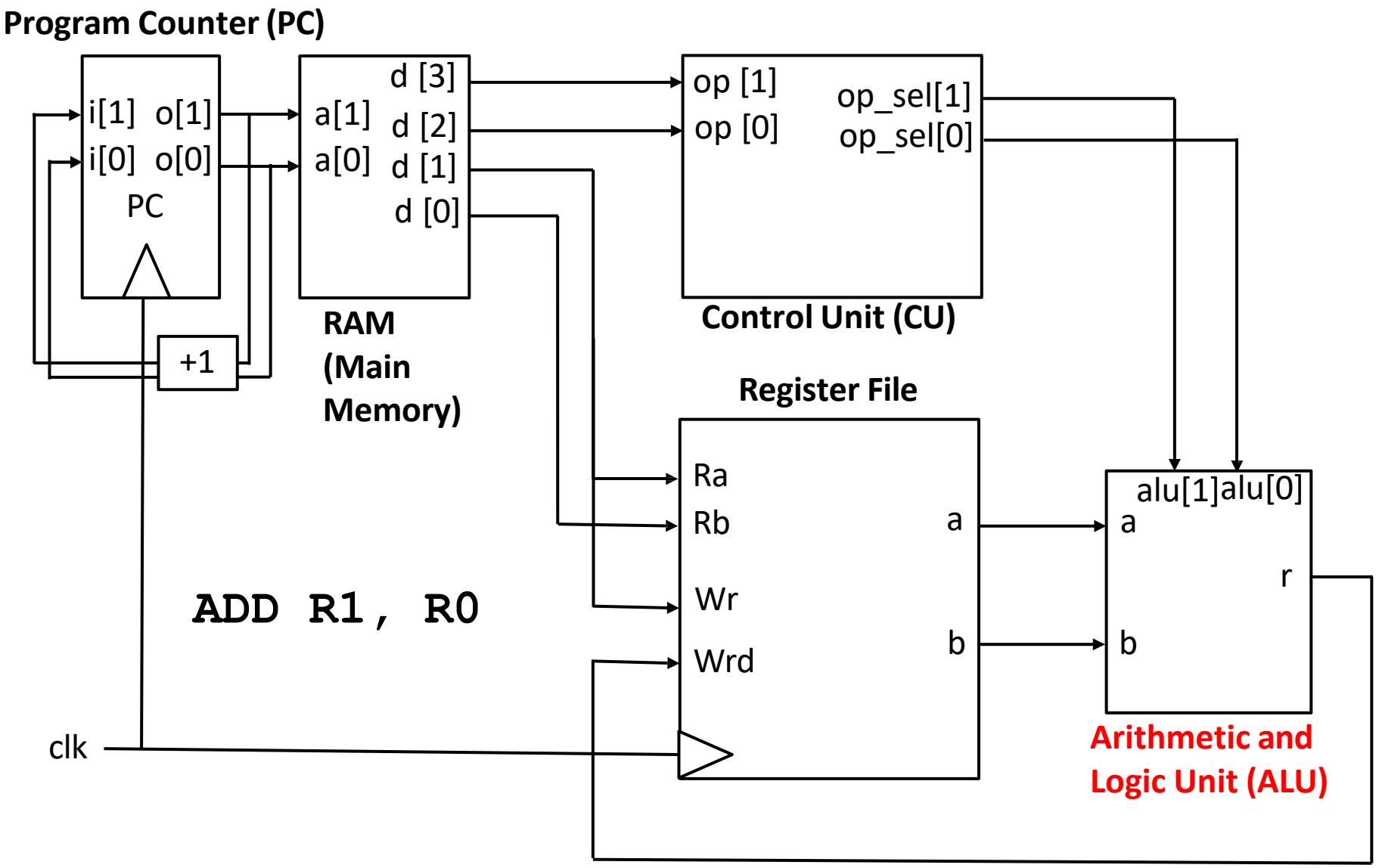


Figure: 1-bit CPU

1. Program Counter will have address of next instruction to be executed in current clock cycle.
2. Address in PC will be sent to RAM to retrieve instruction.
3. Instruction will be decoded by control unit and will select registers and/or immediate values.
4. Data within registers and/or immediate values will be sent to **Arithmetic and Logic Unit (ALU)** to perform operations.
5. The **ALU** will perform operation and result will be sent to the register to be written.
6. Finally, PC will be incremented to point to the next instruction in next clock cycle.

ALU in CPU

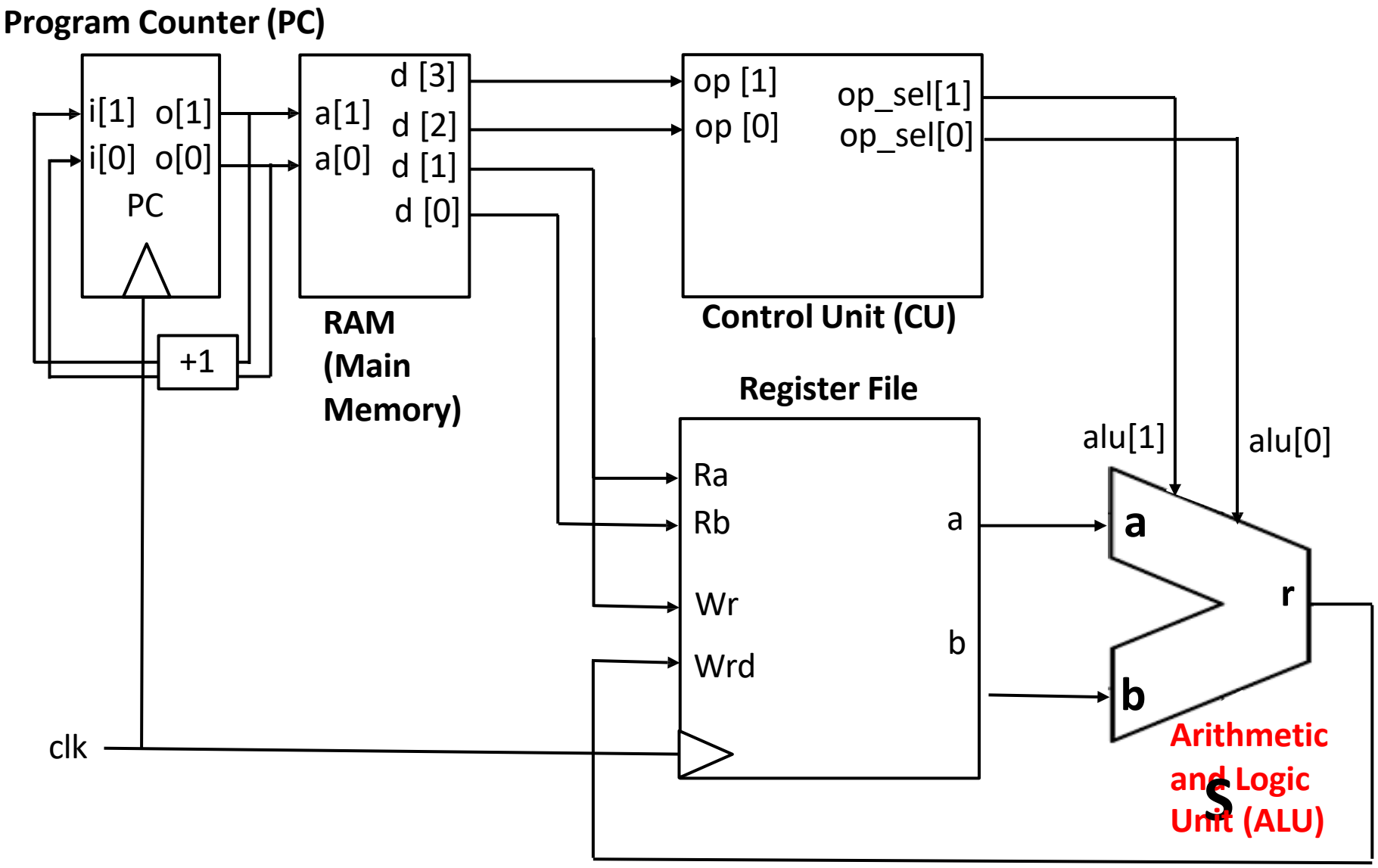


Figure: 1-bit CPU

1. Program Counter will have address of next instruction to be executed in current clock cycle.
2. Address in PC will be sent to RAM to retrieve instruction.
3. Instruction will be decoded by control unit and will select registers and/or immediate values.
4. Data within registers and/or immediate values will be sent to **Arithmetic and Logic Unit (ALU)** to perform operations.
5. The **ALU** will perform operation and result will be sent to the register to be written.
6. Finally, PC will be incremented to point to the next instruction in next clock cycle.

ALU Basics

ALU

Arithmetic Logic Unit (ALU) is a combinational digital circuit that performs arithmetic and bitwise operations on integer binary numbers. It is a fundamental building block of Central Processing Unit (CPU) of computers.

The inputs to an ALU are the data to be operated on, called operands, and a code indicating the operation to be performed. The ALU's output is the result of the performed operation.

In many designs, the ALU also has status/FLAG inputs or outputs, or both, which convey information about a previous operation or the current operation, respectively, between the ALU and external status/FLAG registers.

Functions of ALU

a. **Arithmetic operations:**

1. **Add:** **`A`** and **`B`** are summed and the sum appears at **`Y`** and **`carry-out`**.
2. **Add with carry:** **`A`**, **`B`** and **`carry-in`** are summed and the sum appears at **`Y`** and **`carry-out`**.
3. **Subtract:** **`B`** is subtracted from **`A`** (or vice versa) and the difference appears at **`Y`** and **`carry-out`**. For this function, carry-out is effectively a "borrow" indicator. This operation may also be used to compare the magnitudes of **`A`** and **`B`**; in such cases the **`Y`** output may be ignored by the processor, which is only interested in the status bits (particularly zero and negative) that result from the operation.
4. **Subtract with borrow:** **`B`** is subtracted from **`A`** (or vice versa) with borrow (**`carry-in`**) and the difference appears at **`Y`** and **`carry-out`** (borrow out).
5. **Two's complement (negate):** **`A`** (or **`B`**) is subtracted from zero and the difference appears at **`Y`**.
6. **Increment:** **`A`** (or **`B`**) is increased by one and the resulting value appears at **`Y`**.
7. **Decrement:** **`A`** (or **`B`**) is decreased by one and the resulting value appears at **`Y`**.

Functions of ALU

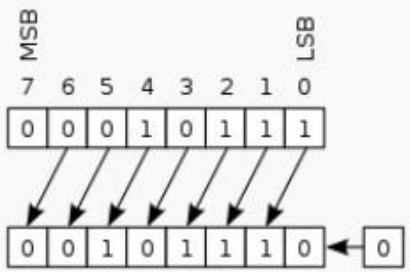
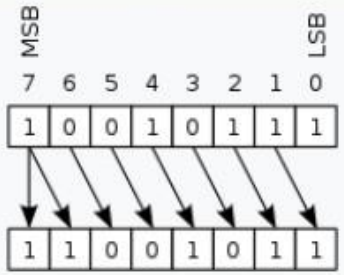
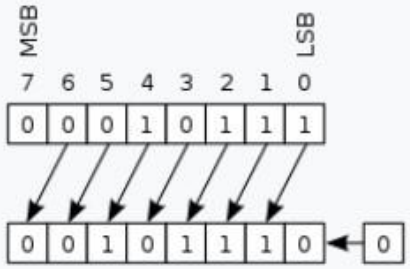
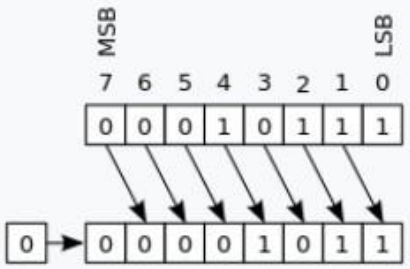
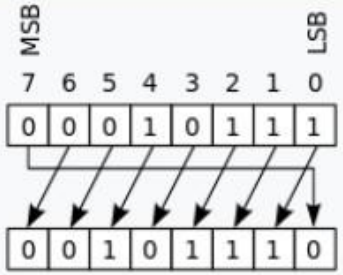
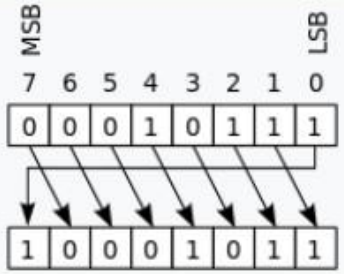
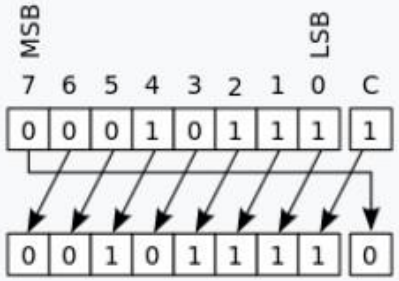
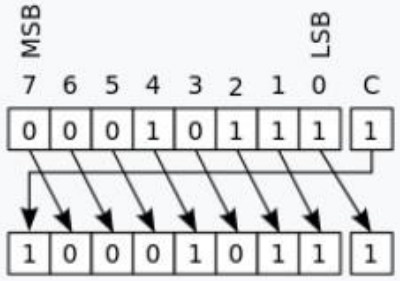
b. **Bitwise logical operations:**

1. **AND:** the bitwise AND of **`A`** and **`B`** appears at **`Y`**.
2. **OR:** the bitwise OR of **`A`** and **`B`** appears at **`Y`**.
3. **Exclusive-OR:** the bitwise XOR of **`A`** and **`B`** appears at **`Y`**.
4. **Ones' complement:** all bits of **`A`** (or **`B`**) are inverted and appear at **`Y`**.

c. **Bit shift operations:** ALU shift operations cause operand **`A`** (or **`B`**) to shift left or right (depending on the opcode) and the shifted operand appears at **`Y`**.

1. **Arithmetic shift:** The operand is treated as a two's complement integer, meaning that the most significant bit is a "sign" bit and is preserved.
2. **Logical shift:** A logic zero is shifted into the operand. This is used to shift unsigned integers.
3. **Rotate:** The operand is treated as a circular buffer of bits so its least and most significant bits are effectively adjacent.
4. **Rotate through carry:** The carry bit and operand are collectively treated as a circular buffer of bits.

Functions of ALU

Type	Left	Right
Arithmetic shift		
Logical shift		
Rotate		
Rotate through carry		

FPU

A floating-point unit (FPU, colloquially a math coprocessor) is a part of a computer system specially designed to carry out operations on floating-point numbers. Typical operations are addition, subtraction, multiplication, division, and square root.

In general-purpose computer architectures, one or more FPUs may be integrated as execution units within the Central Processing Unit (CPU). However, many embedded processors/microcontrollers do not have hardware support for floating-point operations.

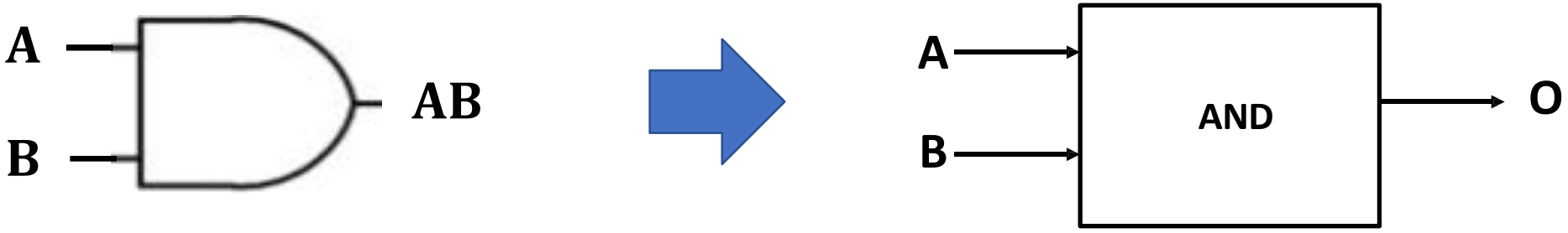
When a CPU is executing a program that calls for a floating-point operation, there are three ways to carry it out:

1. A floating-point unit emulator (A floating-point library/Software).
2. Add-on FPU (Coprocessor/Not inside Processor).
3. Integrated FPU (Inside Processor).

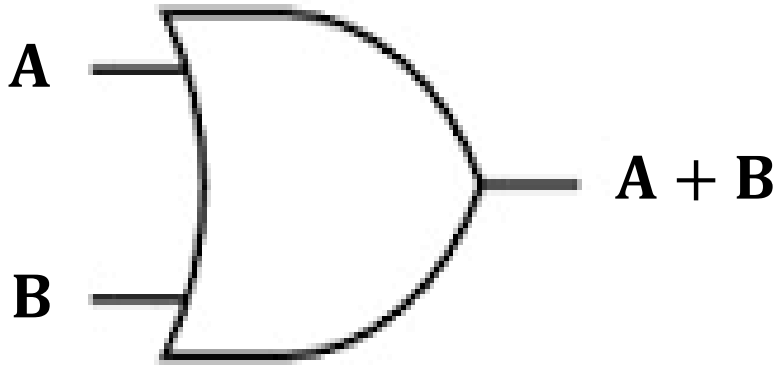
Example 1: 1-bit ALU design

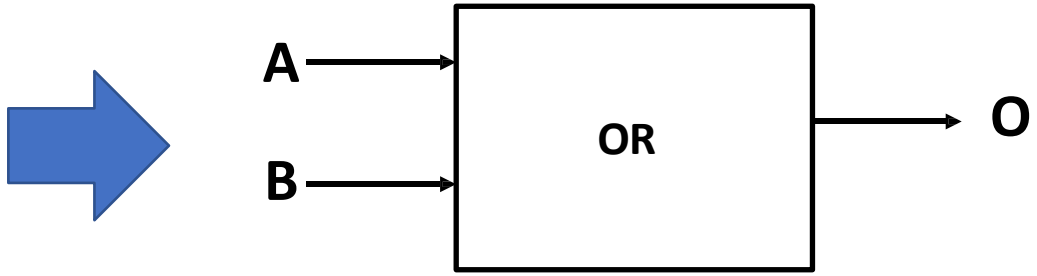
s_1

1-bit AND gate

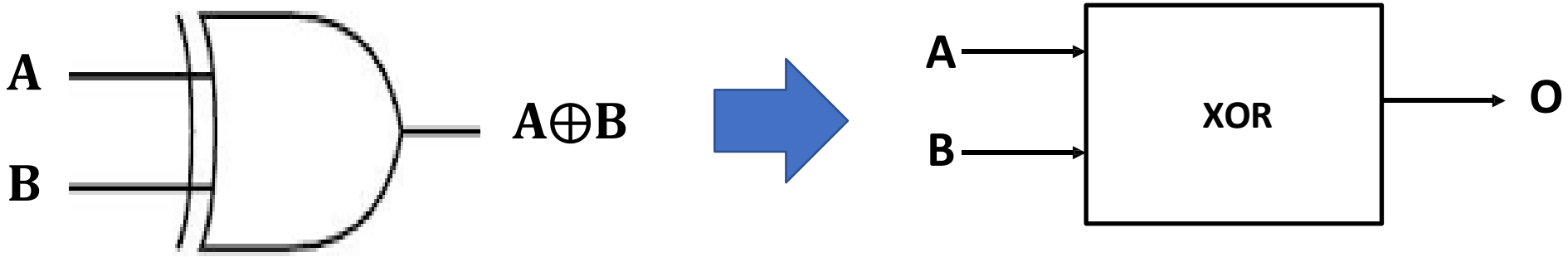


1-bit OR gate

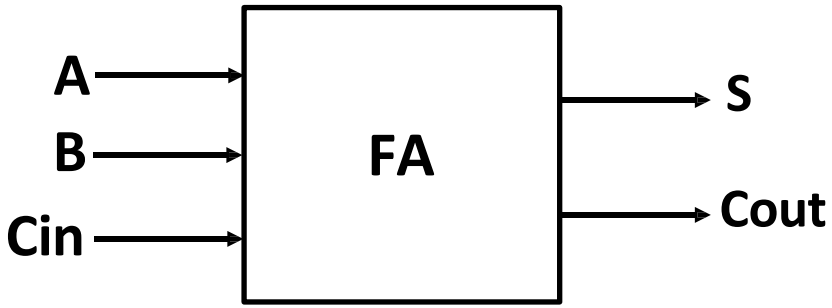




1-bit XOR gate



1-bit Adder

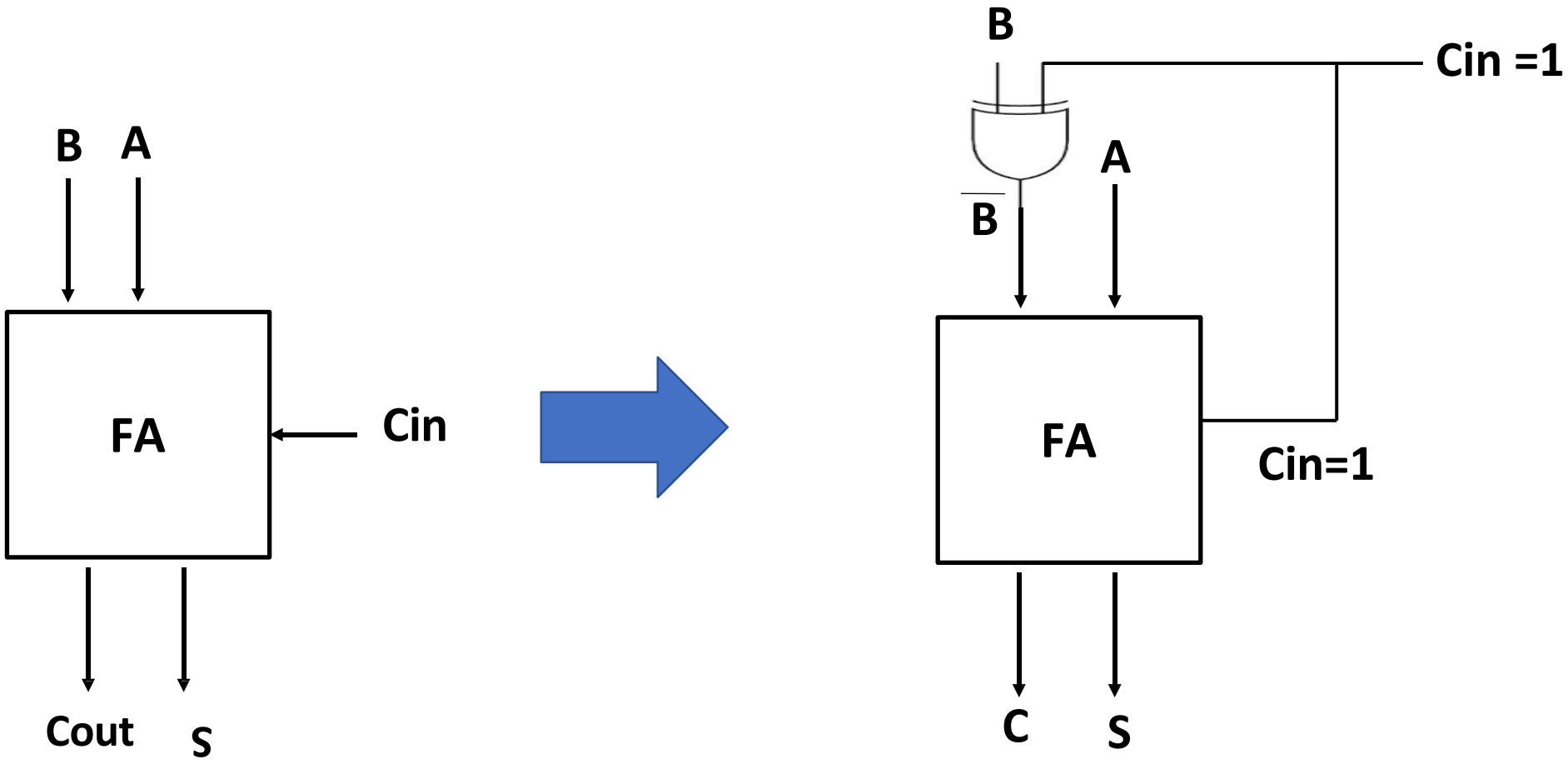


A	B	Cin	Cout	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

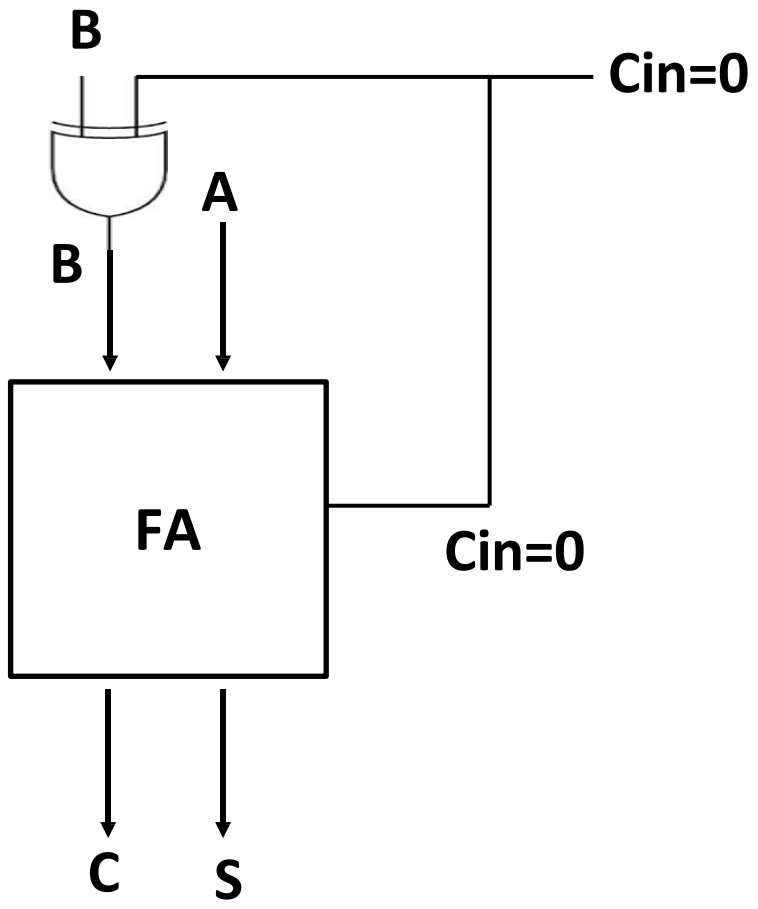
$$S = A\,\overline{B}\,\overline{Cin} + \overline{A}\,B\,\overline{Cin} + \overline{A}\,\overline{B}\,Cin + A\,B\,Cin$$

$$Cout = A\,B + A\,C + B\,C$$

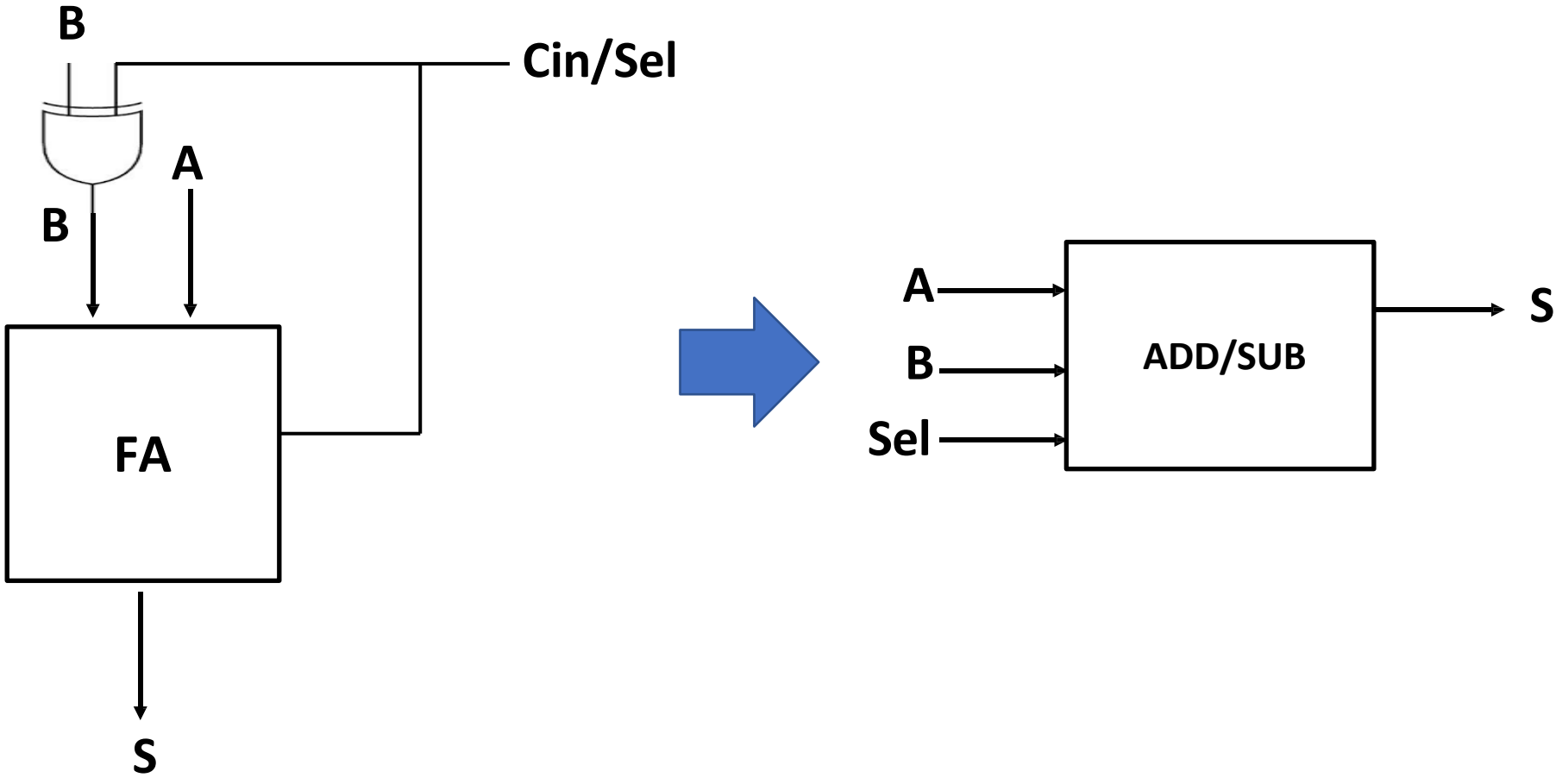
1-bit Subtractor



1-bit Adder (Modified)

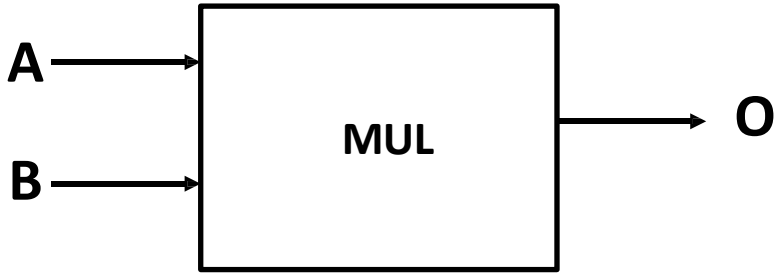


1-bit Adder/Subtractor



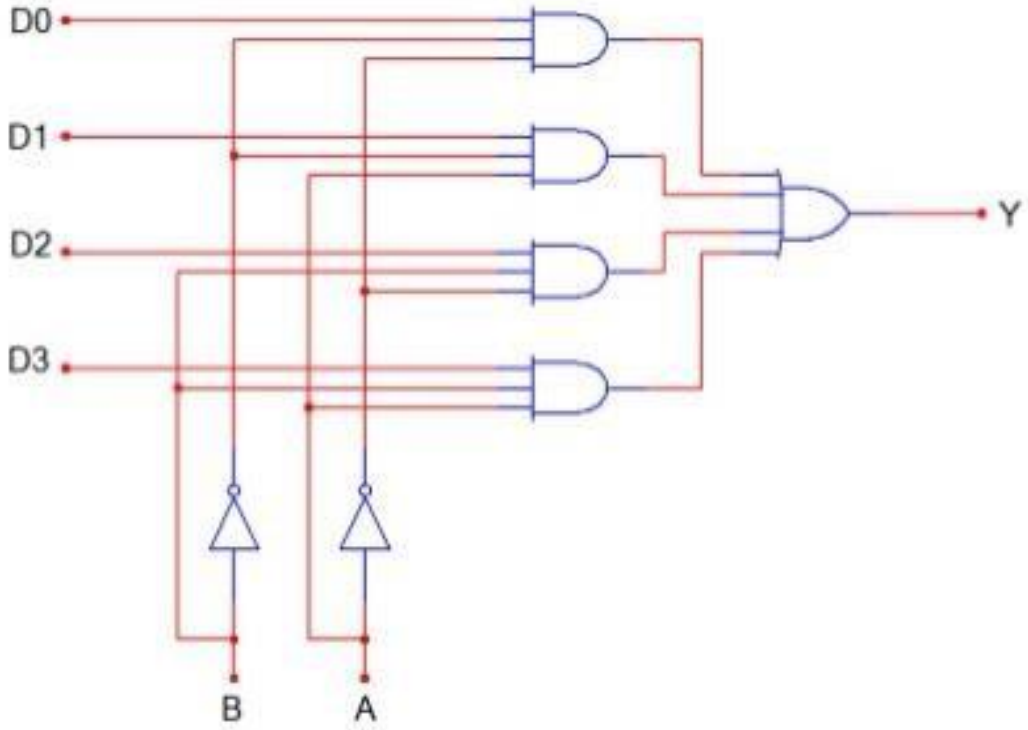
1-bit Multiplier

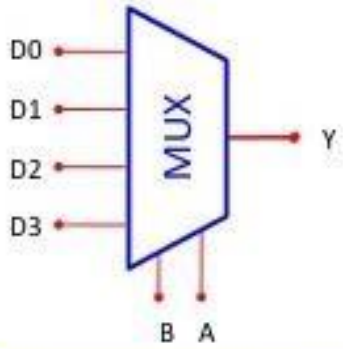
A	B	M
0	* 0	= 0
0	* 1	= 0
1	* 0	= 0
1	* 1	= 1



1-bit Shifter

4-to-1 Multiplexer (MUX)

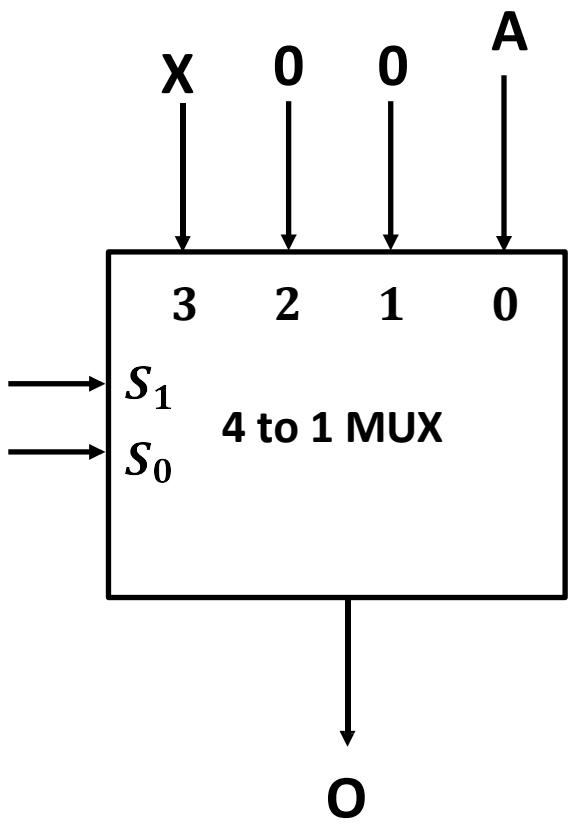




B	A	Y
0	0	D0
0	1	D1
1	0	D2
1	1	D3

$$Y = \overline{A}.\overline{B}.D0 + \overline{A}.B.D1 + A.\overline{B}.D2 + A.B.D3$$

1-bit Shifter

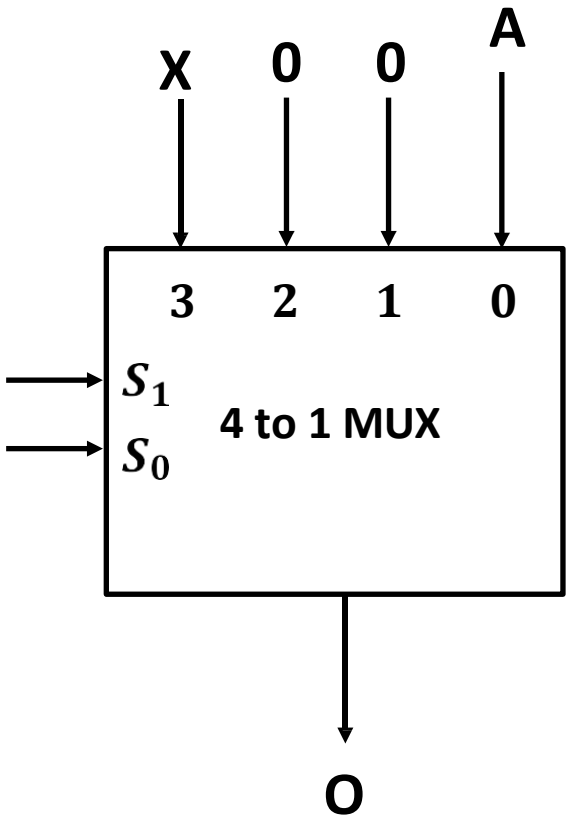


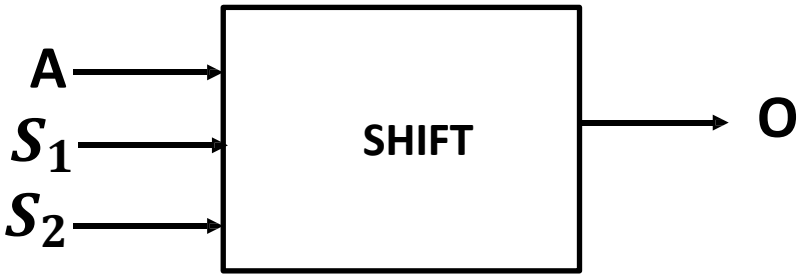
S_1	S_2	Output	Operation
0	0	A	No Shift
0	1	0	Left Shift
1	0	0	Right Shift
1	1	X	X

Input: A
Right shift: 0A
Left shift: A0

Input: 1
Right shift: 01
Left shift: 10

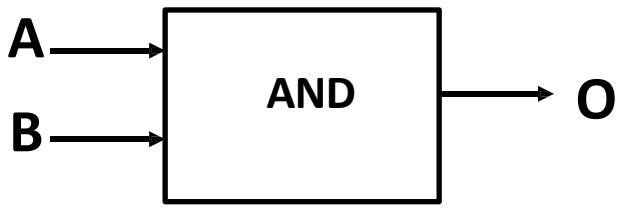
1-bit Shifter

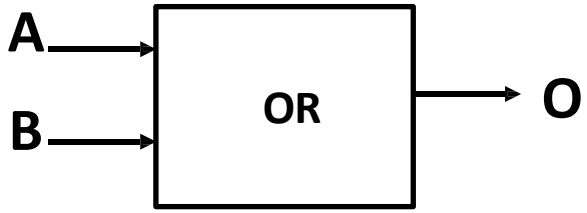


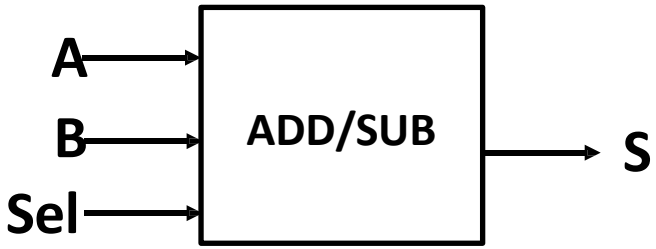


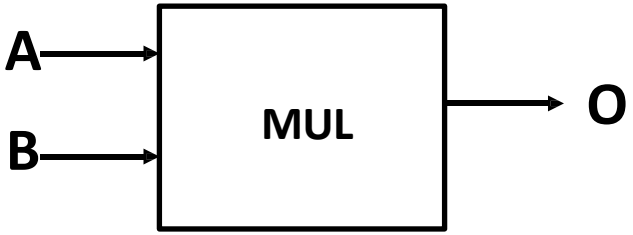
S_1	S_2	Output	Operation
0	0	A	No Shift
0	1	0	Left Shift
1	0	0	Right Shift
1	1	X	X

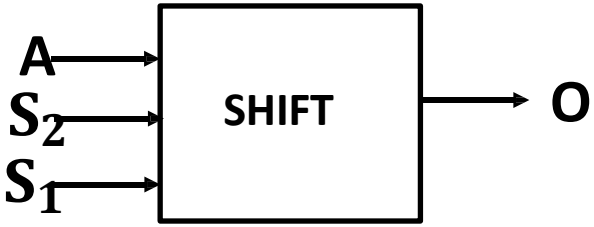
All the circuits so far







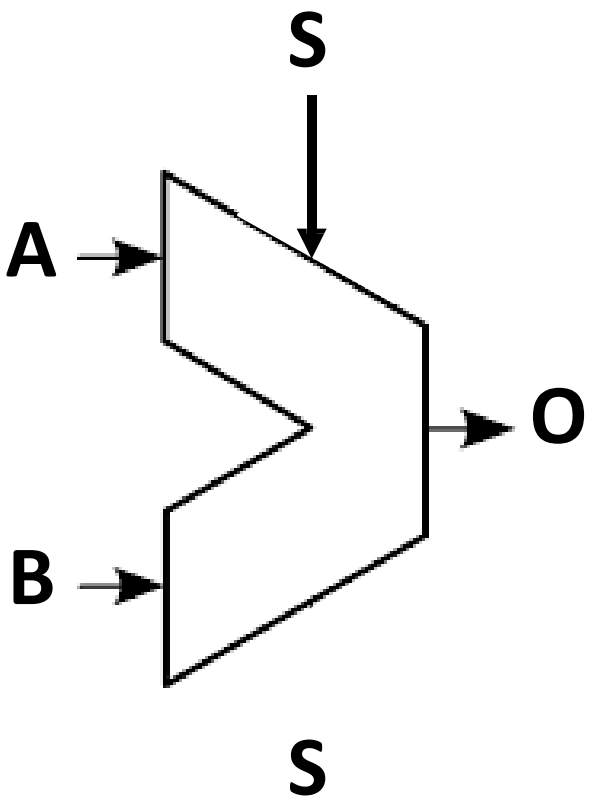




If $\mathbf{sel = 0,\ S = ADD}$
If $\mathbf{sel = 1,\ S = SUB}$

If $\mathbf{S_1 = 0, S_0 = 1, S = LEFT\ SHIFT}$
If $\mathbf{S_1 = 1, S_0 = 0, S = RIGHT\ SHIFT}$

ALU Circuit



1-bit CPU

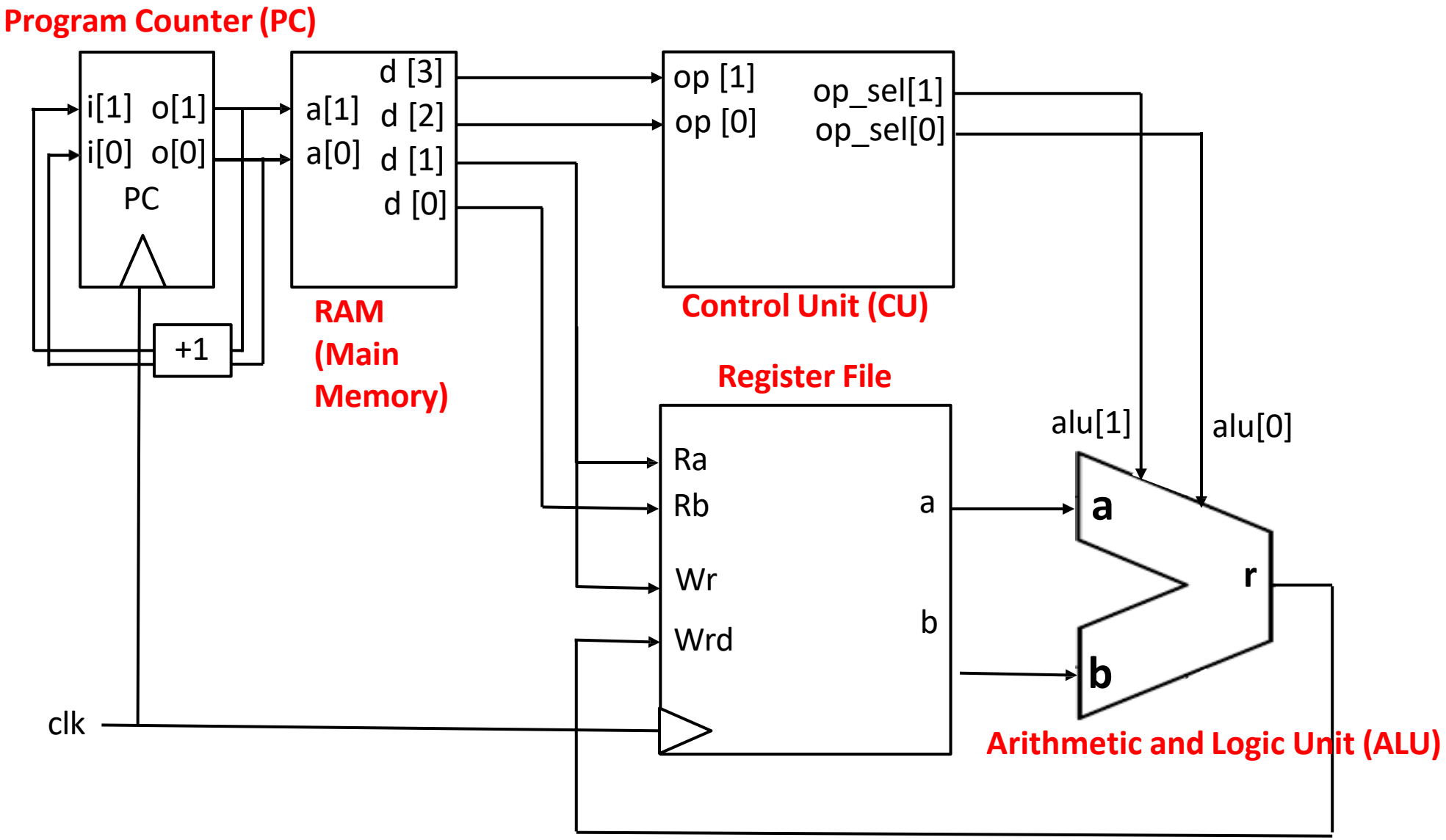
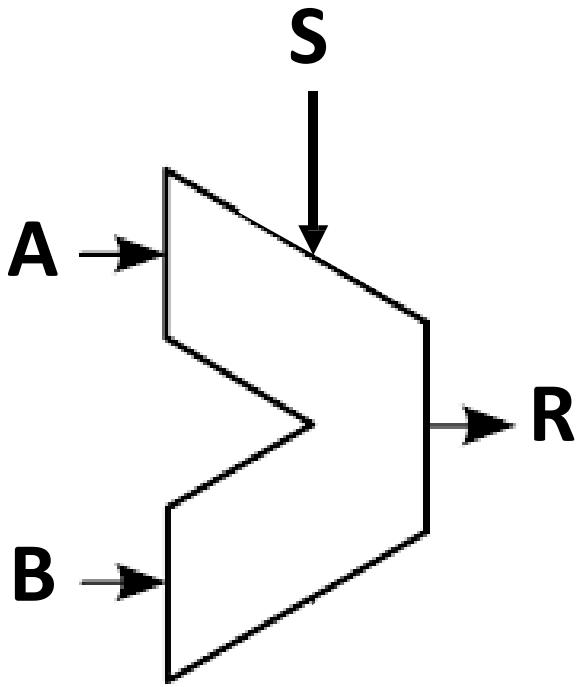


Figure: 1-bit CPU

ALU Circuit



Available Operations:

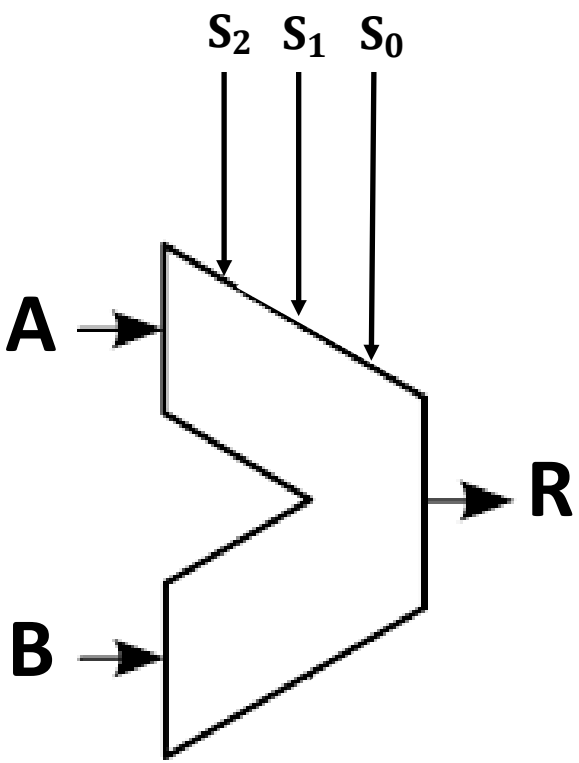
ADD,
SUB,
MUL,
AND,
OR,
XOR,
LEFT SHIFT,
RIGHT SHIFT

All operations will be executed at the same.

Only one operation must be selected by Control Unit.

Total Operations: 8

1-bit ALU Circuit



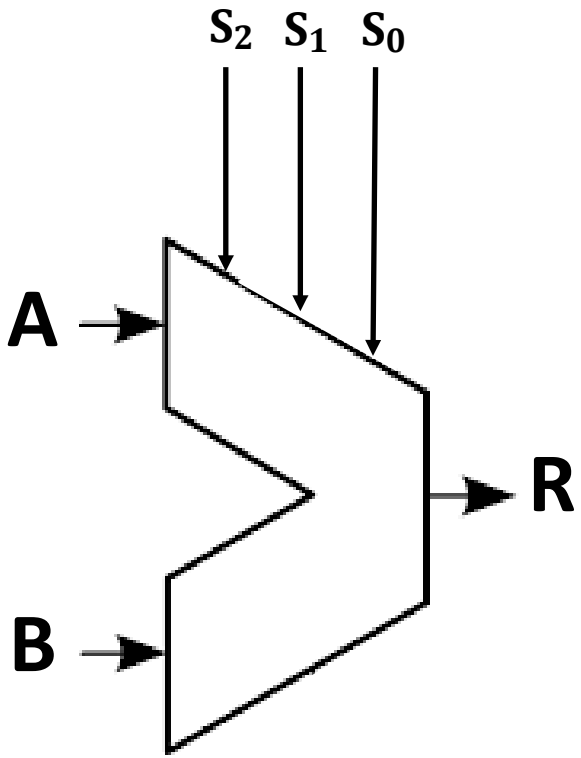
Available Operations:

ADD,
SUB,
MUL,
AND,
OR,
XOR,
LEFT SHIFT,
RIGHT SHIFT

Total Operations: 8

So, selection line must support at least 8 combinations.

1-bit ALU Circuit

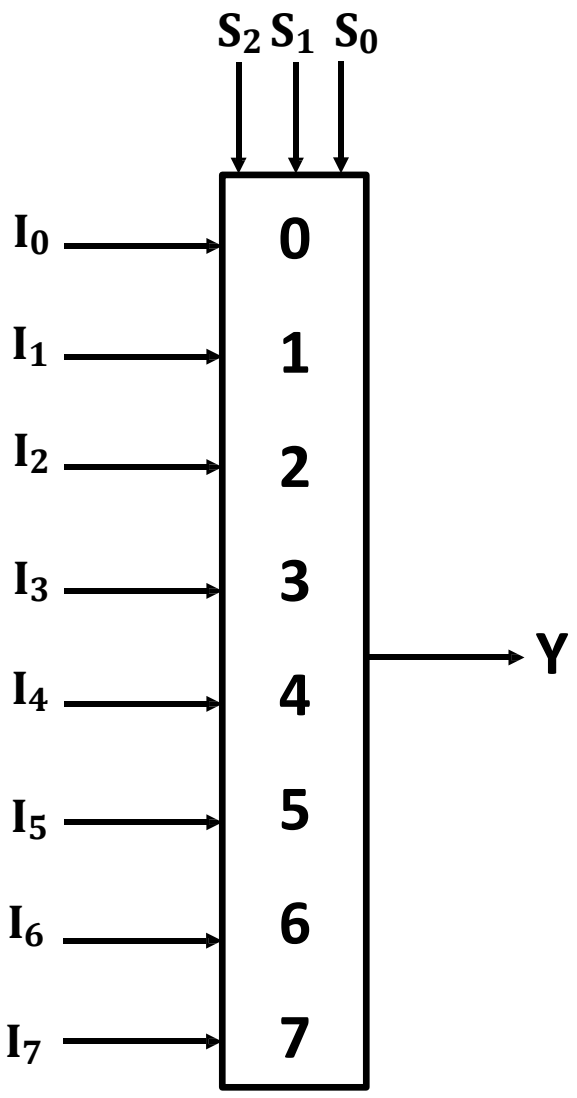


We will need a MUX to select our expected output.

As there are eight operations in total, we will use 8 to 1 MUX

1-bit ALU Circuit

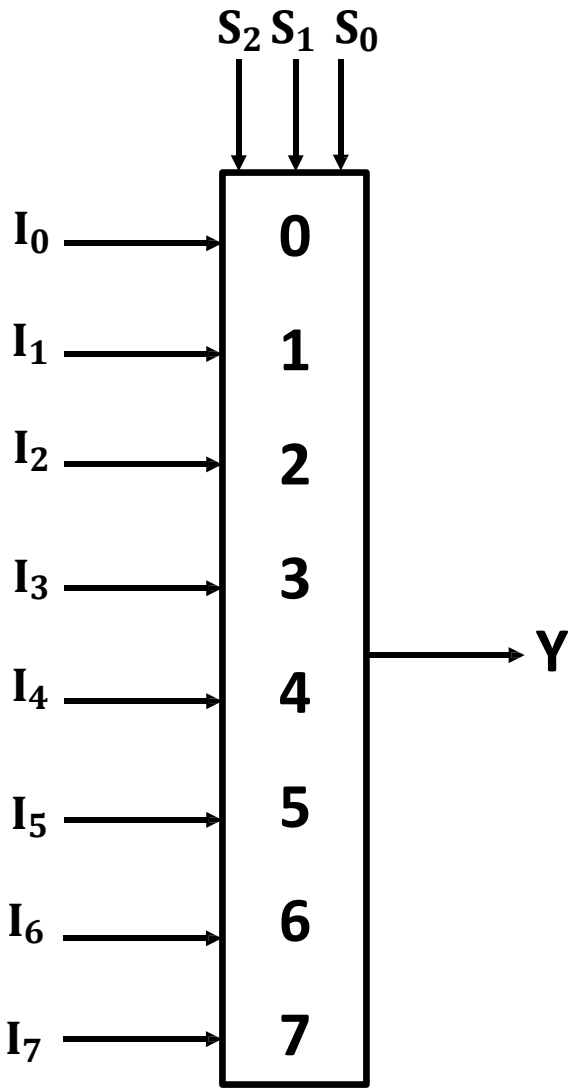
8 to 1 MUX



S_2	S_1	S_0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

$$Y = \overline{S_2}.\overline{S_1}.\overline{S_0}.I_0 + \overline{S_2}.\overline{S_1}.S_0.I_1 + \overline{S_2}.S_1.\overline{S_0}.I_2 + \overline{S_2}.S_1.S_0.I_3 + S_2.\overline{S_1}.\overline{S_0}.I_4 + S_2.\overline{S_1}.S_0.I_5 + S_2.S_1.\overline{S_0}.I_6 + S_2.S_1.S_0.I_7$$

1-bit ALU Circuit

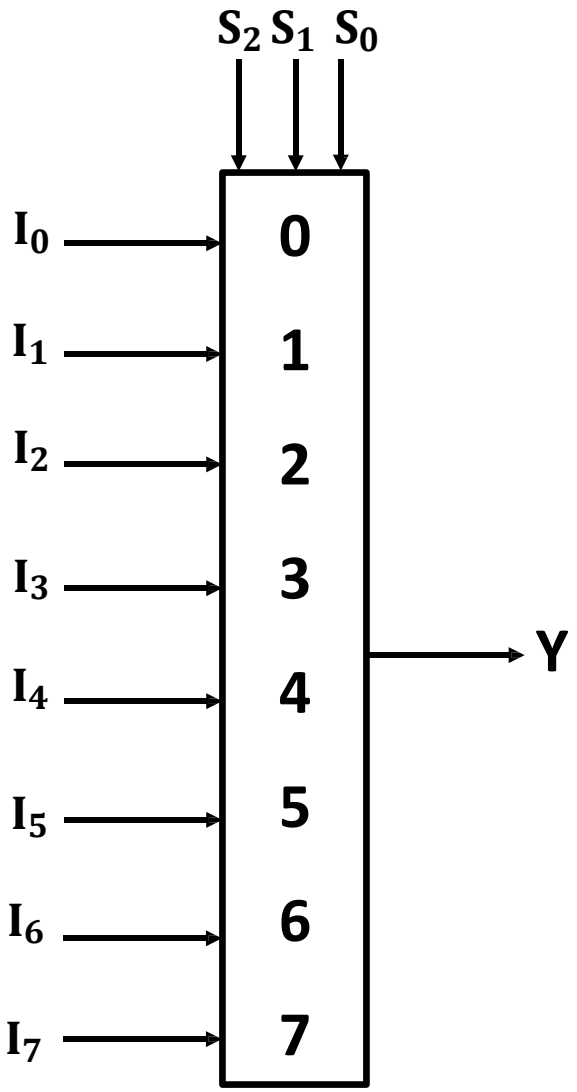


Operation	S_2	S_1	S_0	O
AND	0	0	0	I_0
OR	0	0	1	I_1
XOR	0	1	0	I_2
LEFT SHIFT	0	1	1	I_3
RIGHT SHIFT	1	0	0	I_4
ADD	1	0	1	I_5
SUB	1	1	0	I_6
MUL	1	1	1	I_7

If $\mathbf{sel = 0,\ S = ADD}$
If $\mathbf{sel = 1,\ S = SUB}$

If $\mathbf{S_1 = 0, S_0 = 1, S = LEFT\ SHIFT}$
If $\mathbf{S_1 = 1, S_0 = 0, S = RIGHT\ SHIFT}$

1-bit ALU Circuit

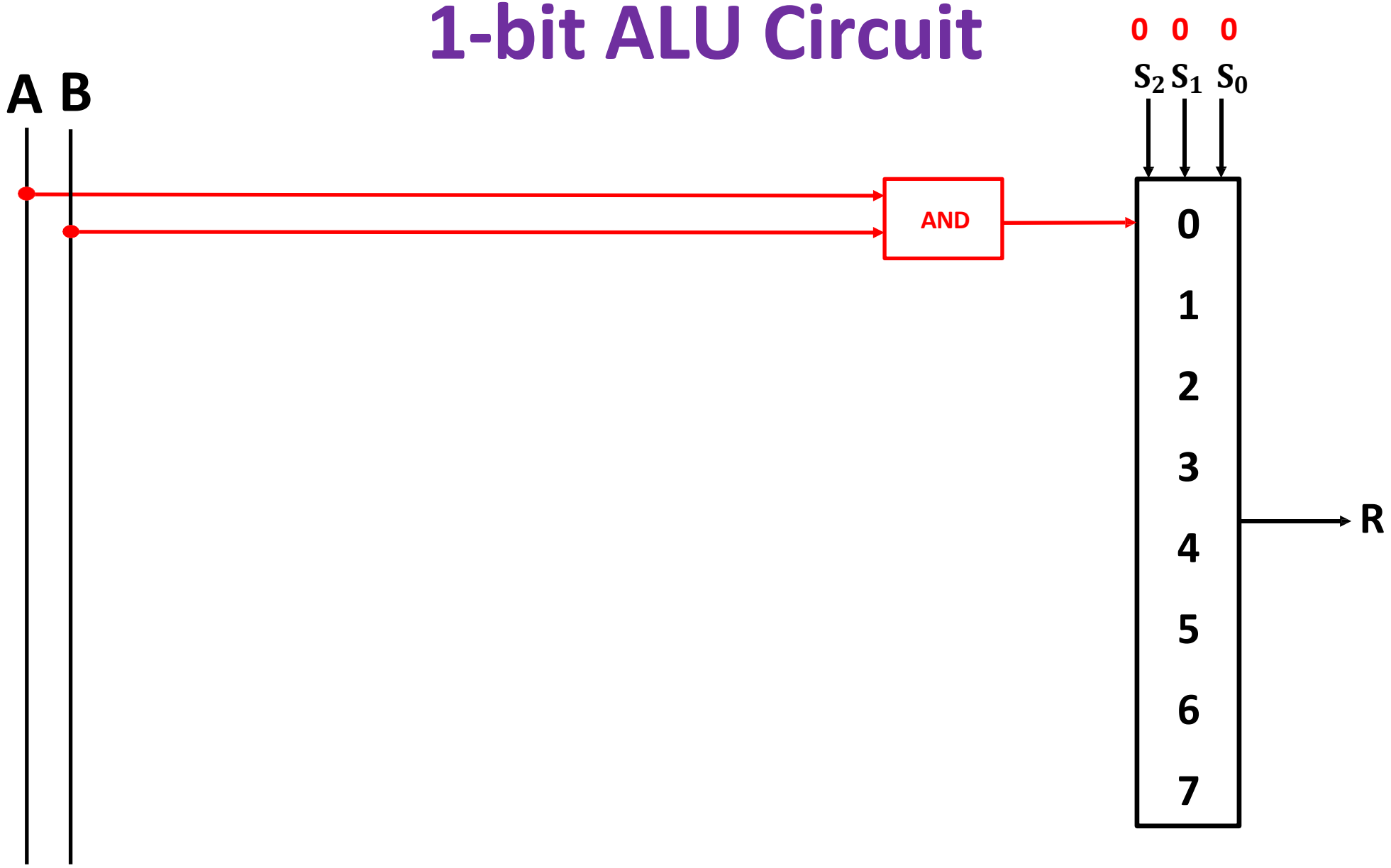


Operation	S_2	S_1	S_0	O
AND	0	0	0	I_0
OR	0	0	1	I_1
XOR	0	1	0	I_2
LEFT SHIFT	0	1	1	I_3
RIGHT SHIFT	1	0	0	I_4
ADD	1	0	1	I_5
SUB	1	1	0	I_6
MUL	1	1	1	I_7

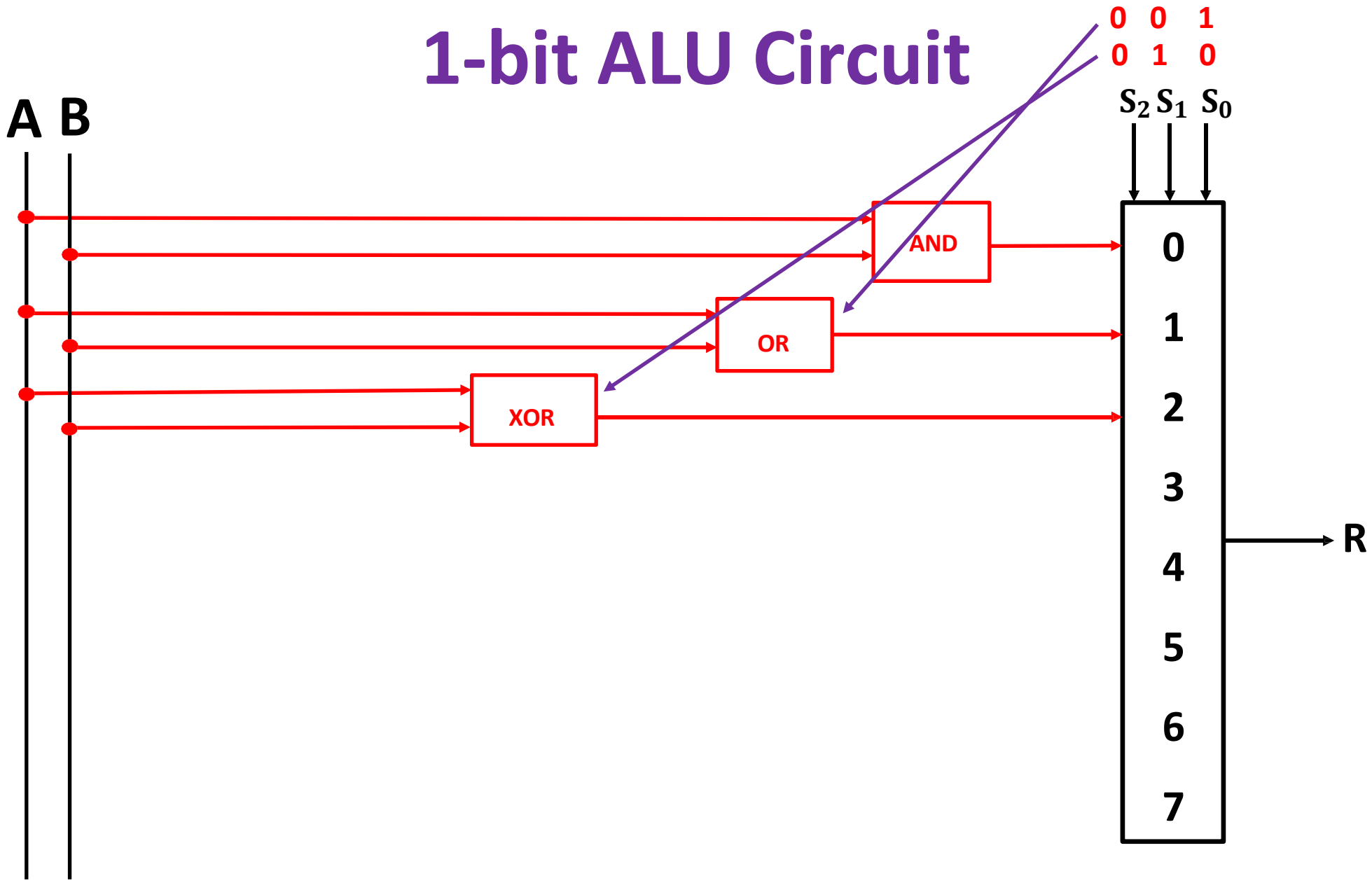
If $\mathbf{sel = 0,\ S = ADD}$
If $\mathbf{sel = 1,\ S = SUB}$

If $\mathbf{S_1 = 0, S_0 = 1, S = LEFT\ SHIFT}$
If $\mathbf{S_1 = 1, S_0 = 0, S = RIGHT\ SHIFT}$

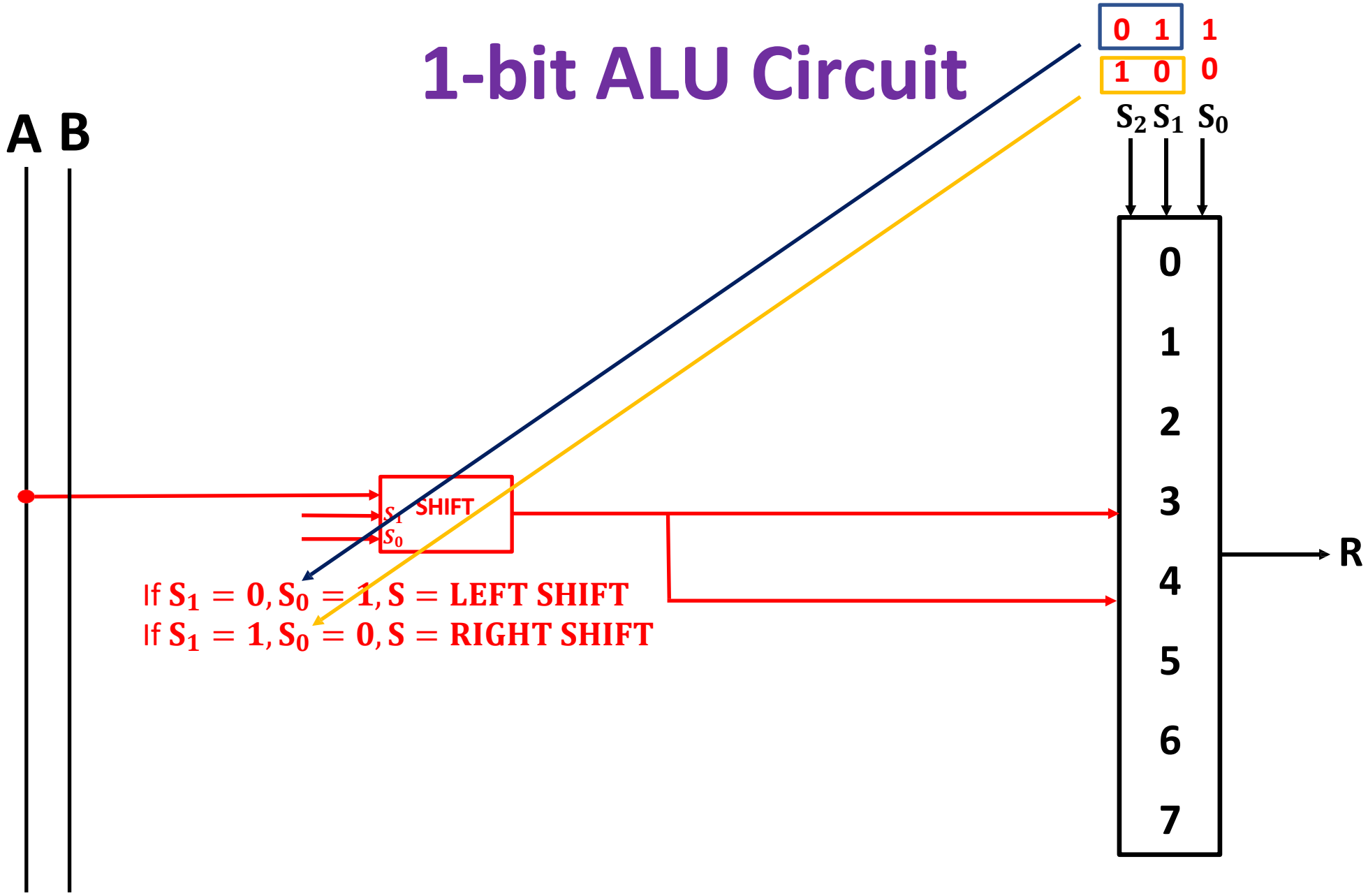
1-bit ALU Circuit



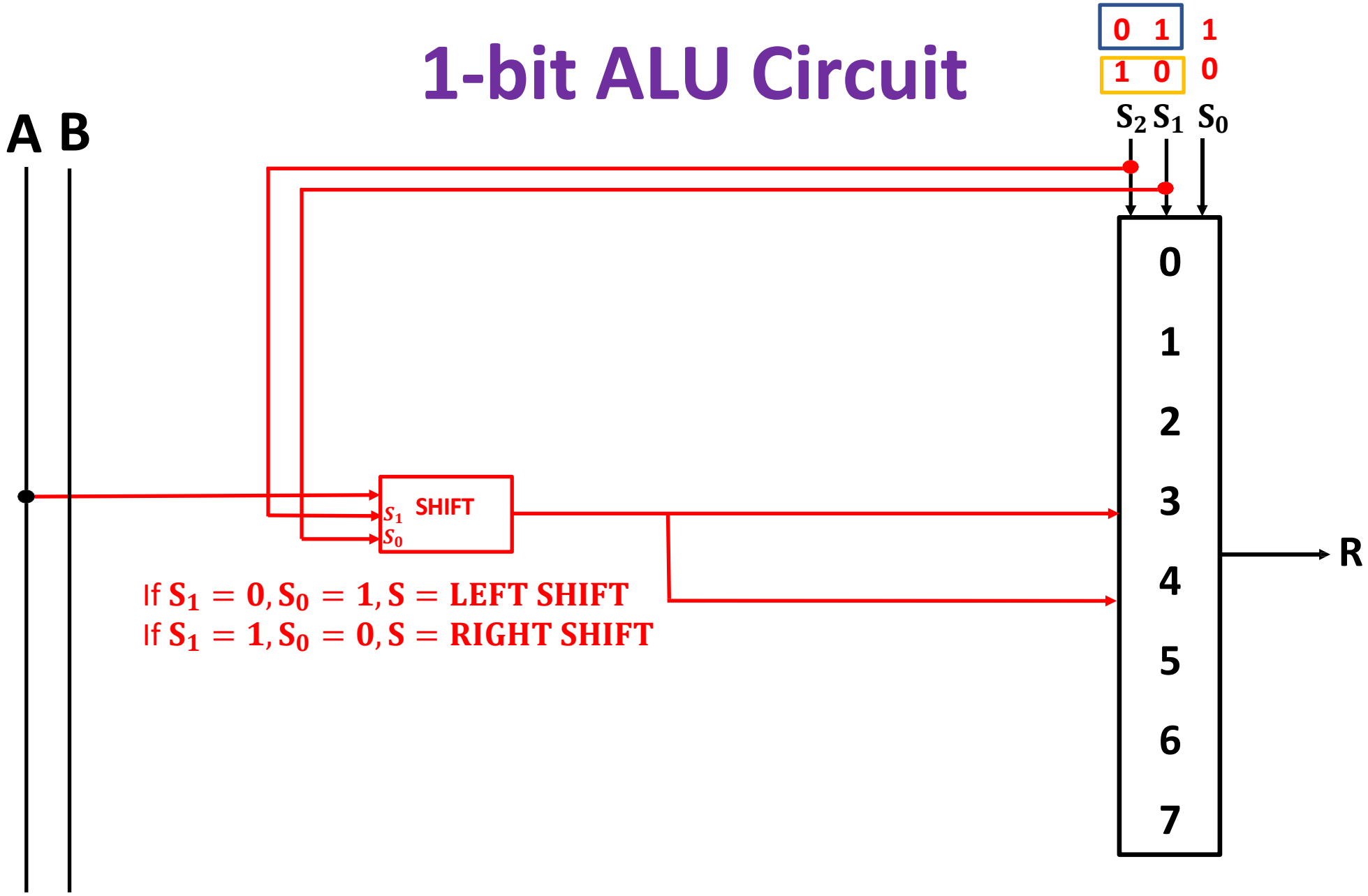
1-bit ALU Circuit



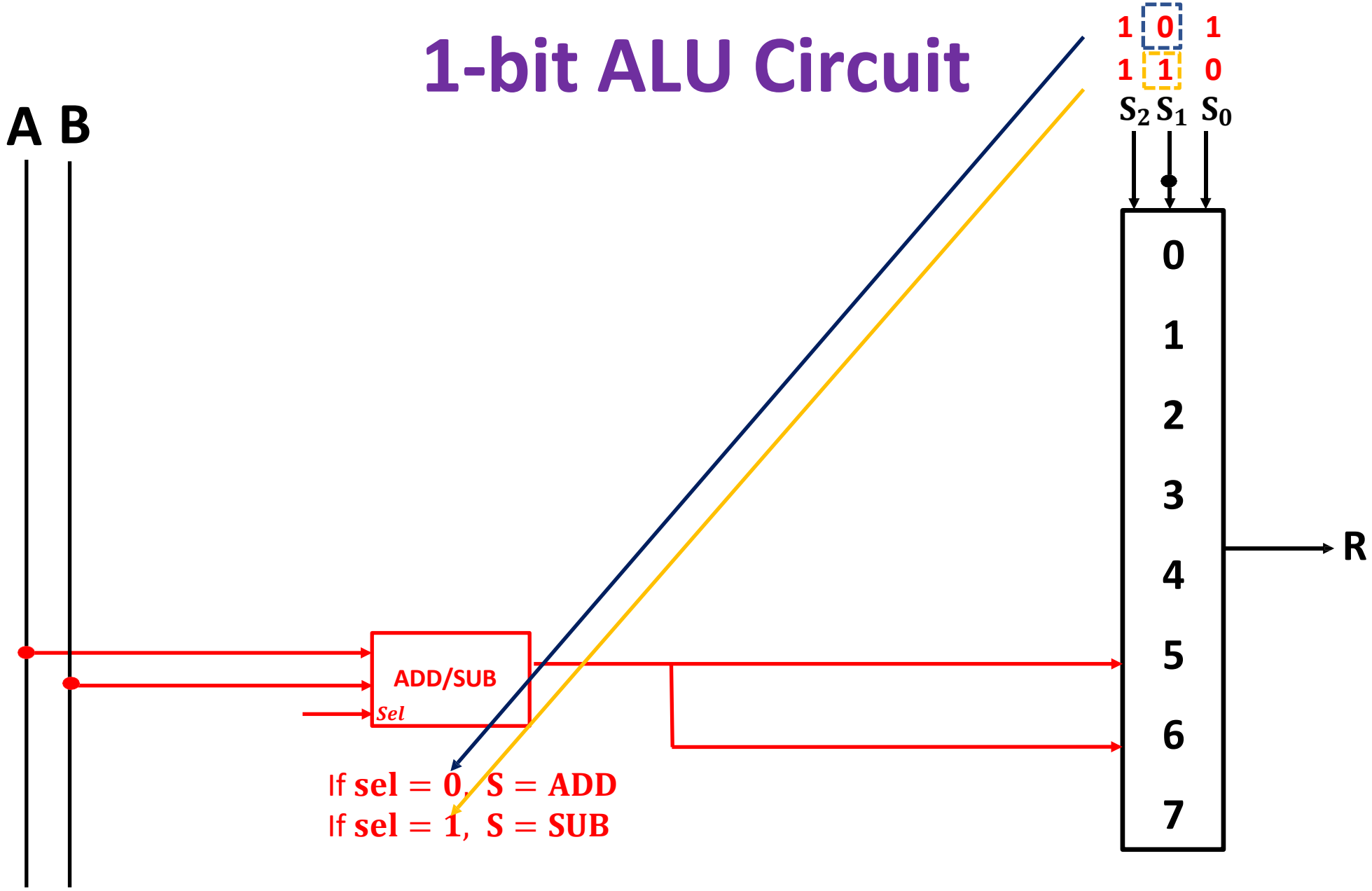
1-bit ALU Circuit



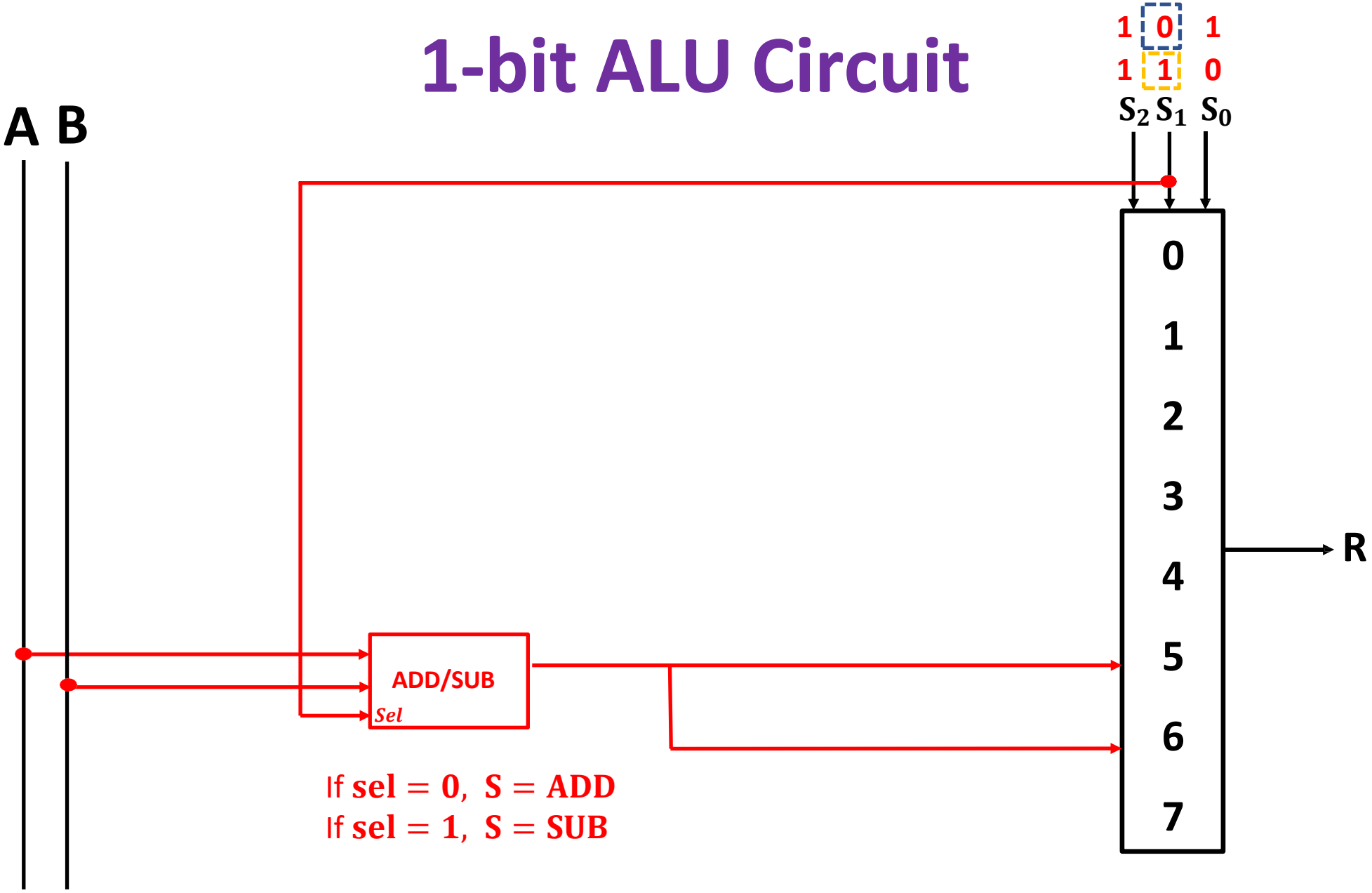
1-bit ALU Circuit



1-bit ALU Circuit

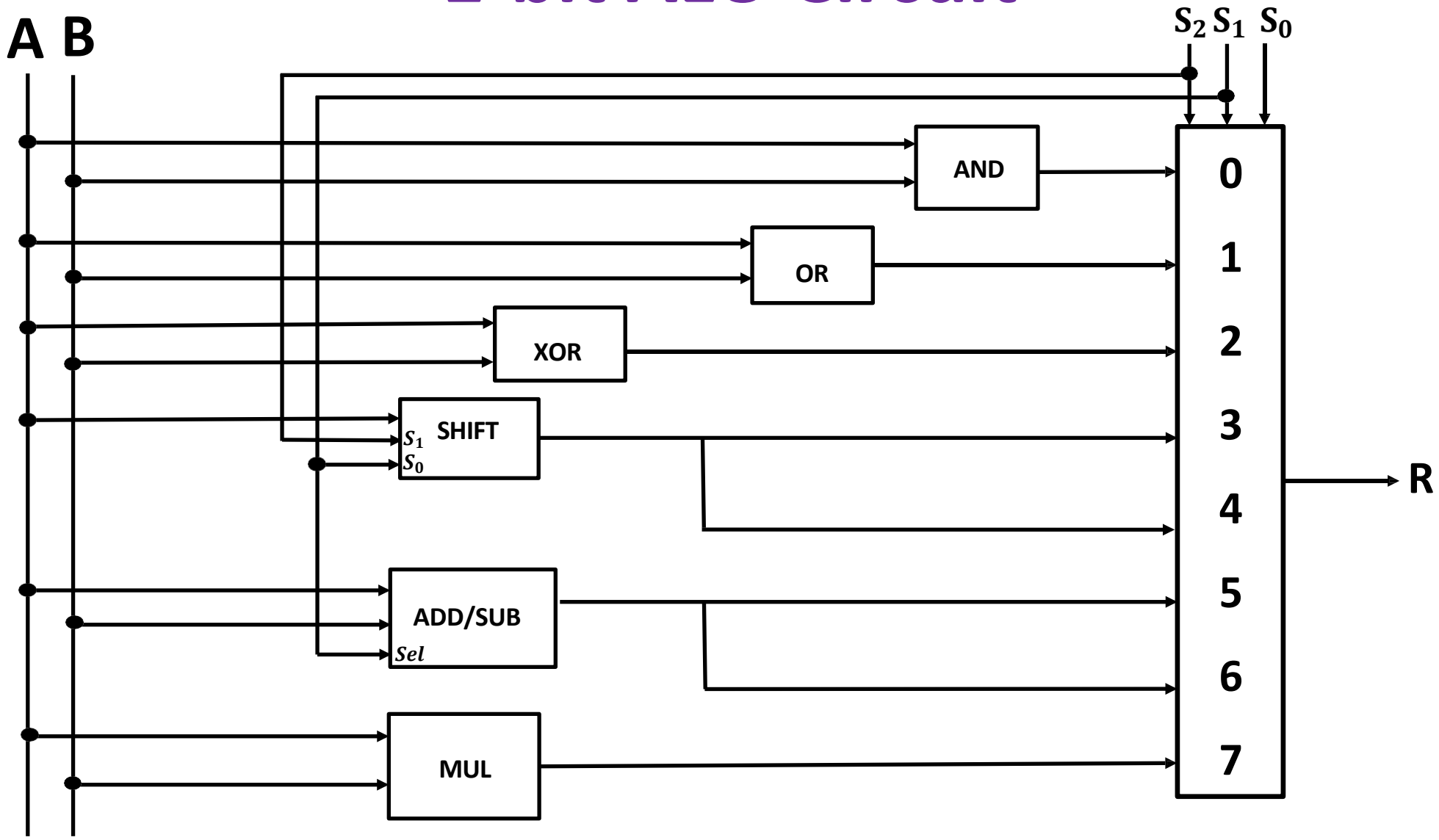


1-bit ALU Circuit



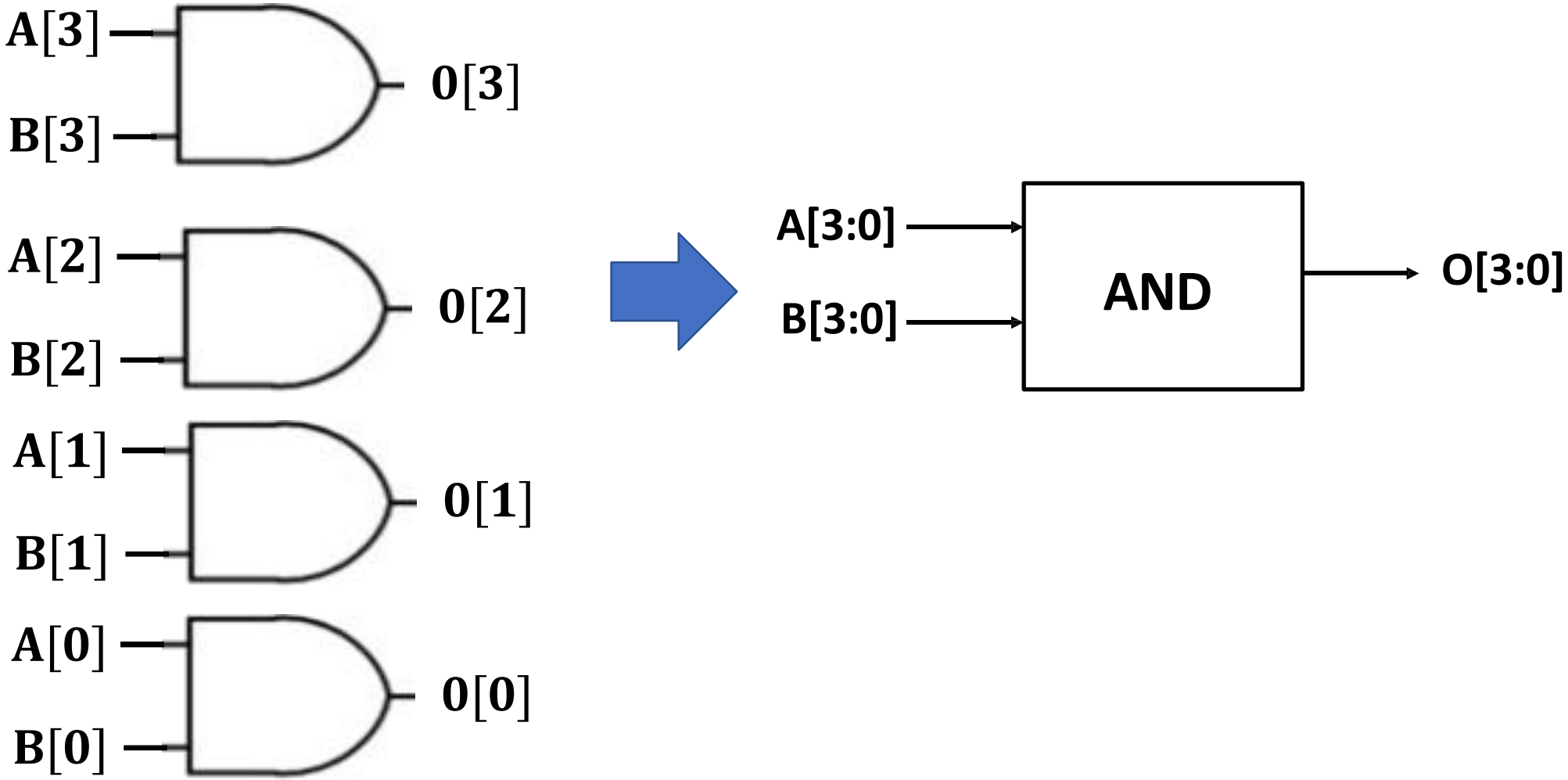
If $\mathbf{sel} = \mathbf{0},\ \mathbf{S} = \mathbf{ADD}$
If $\mathbf{sel} = \mathbf{1},\ \mathbf{S} = \mathbf{SUB}$

1-bit ALU Circuit

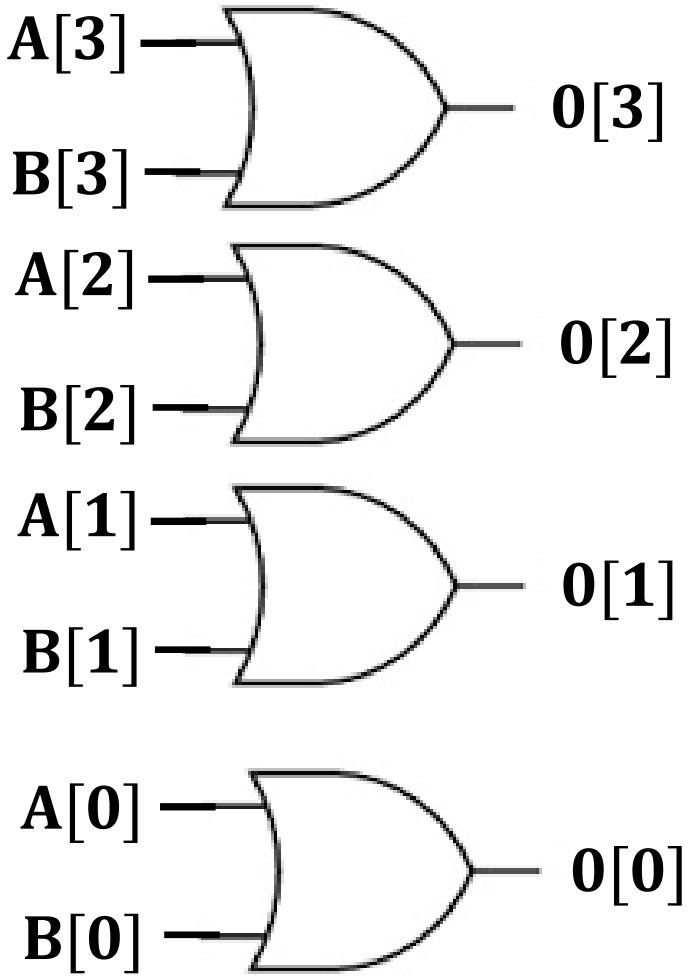


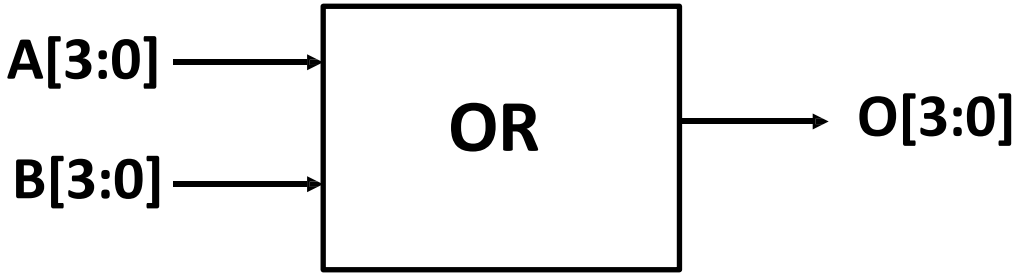
Example 2: 4-bit ALU design

4-bit AND gate

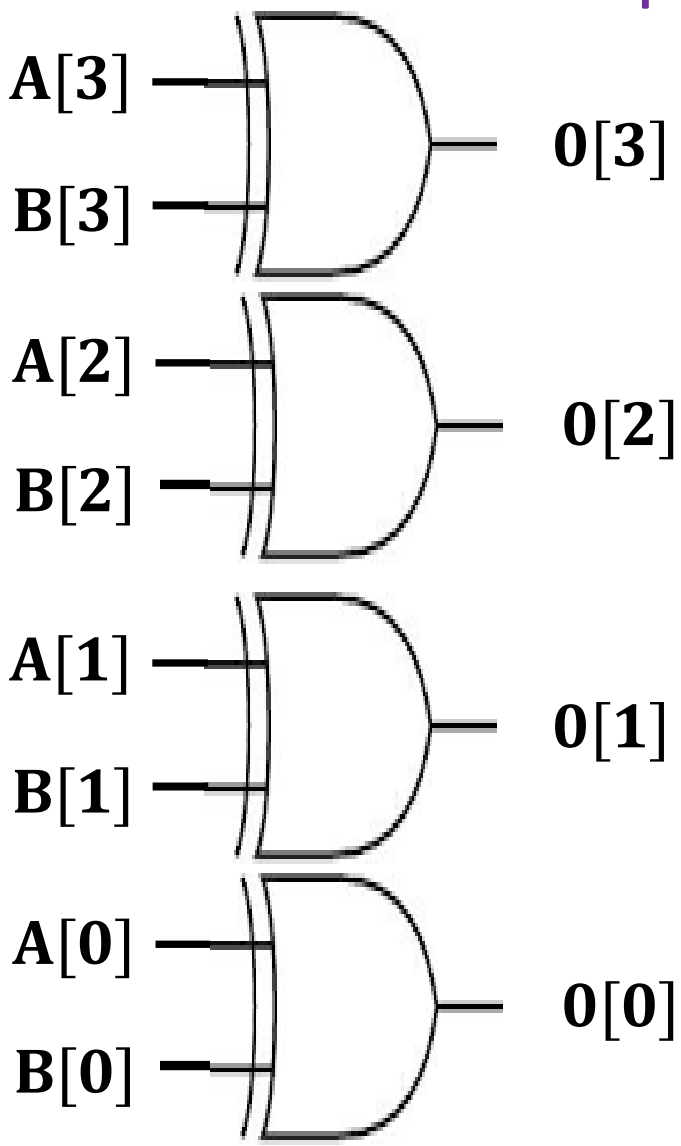


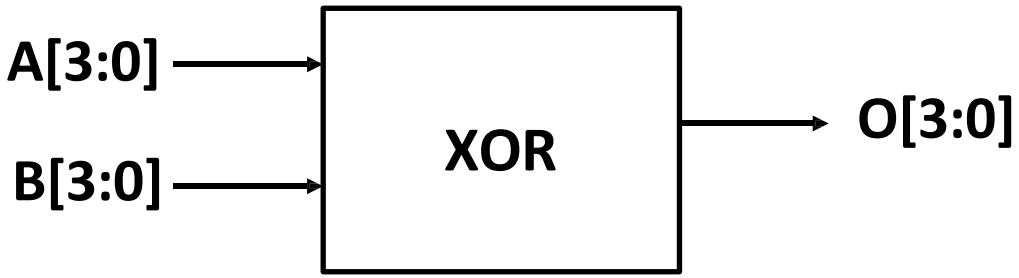
4-bit OR gate



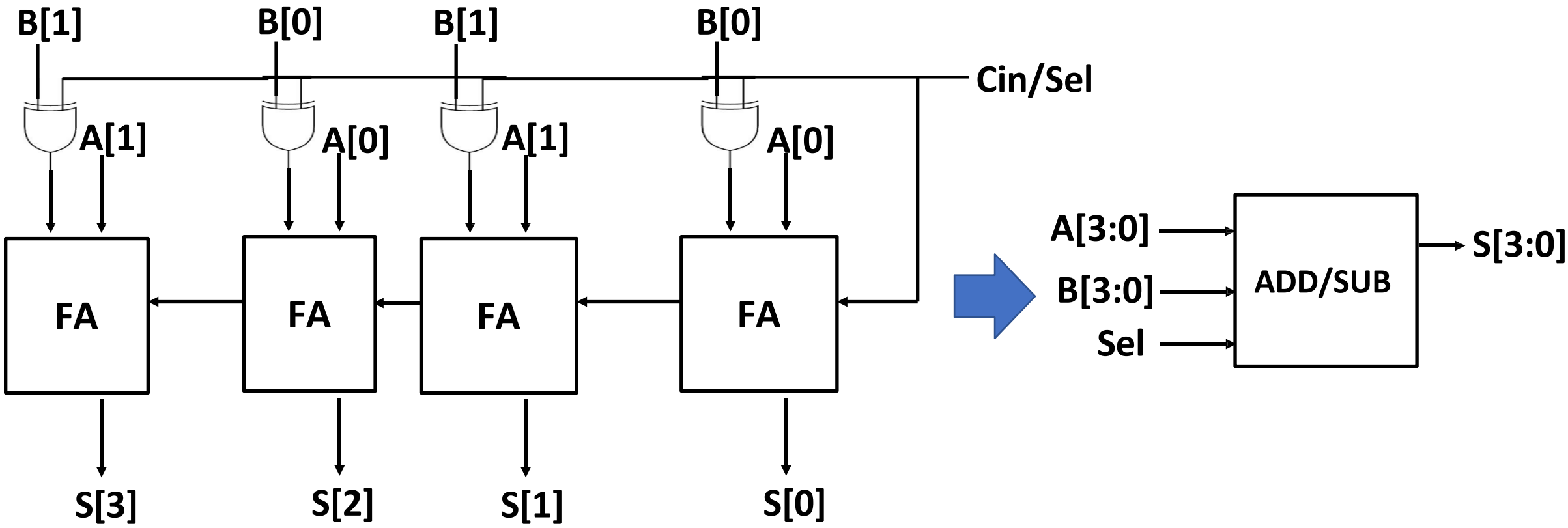


4-bit OR gate



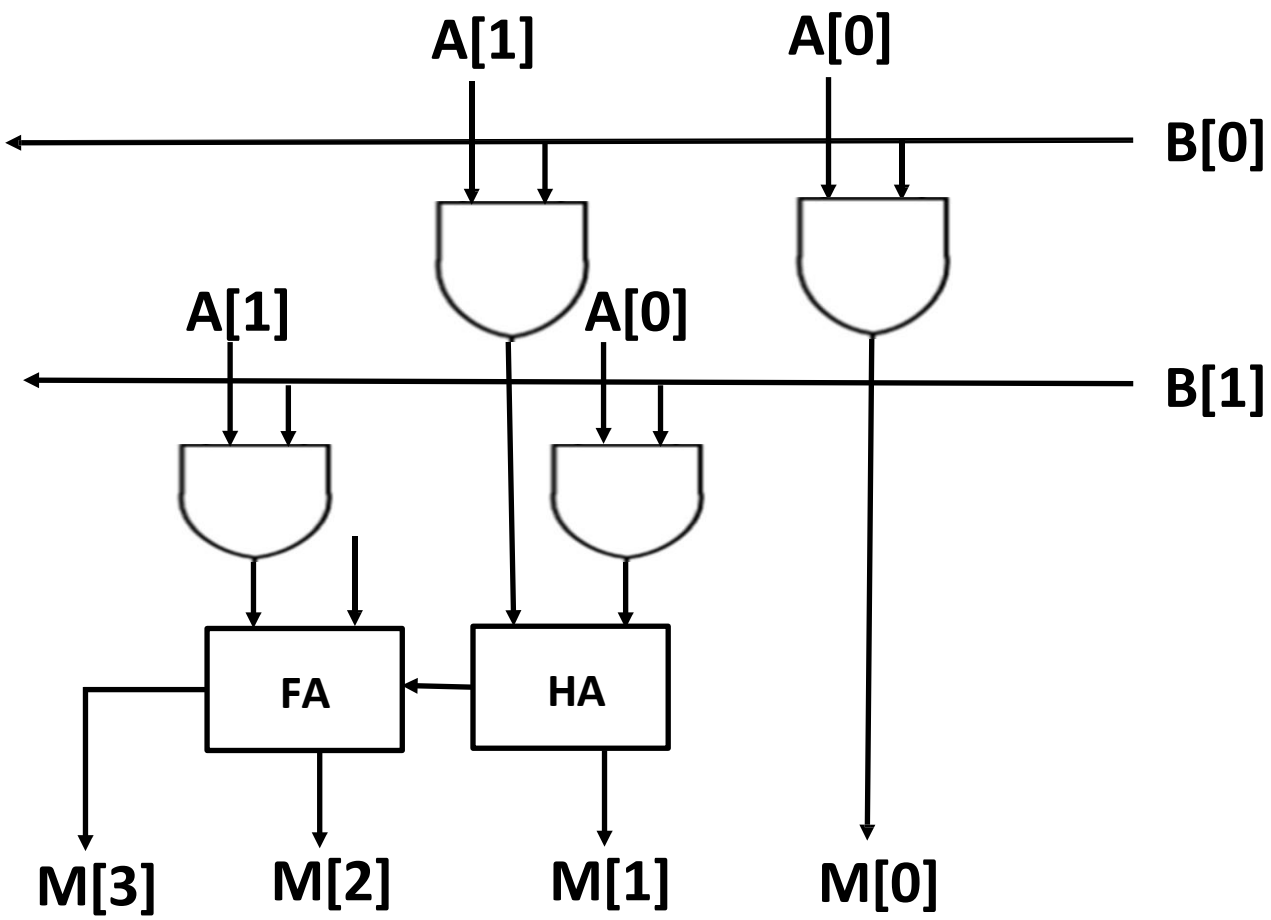


2-bit Adder/Subtractor

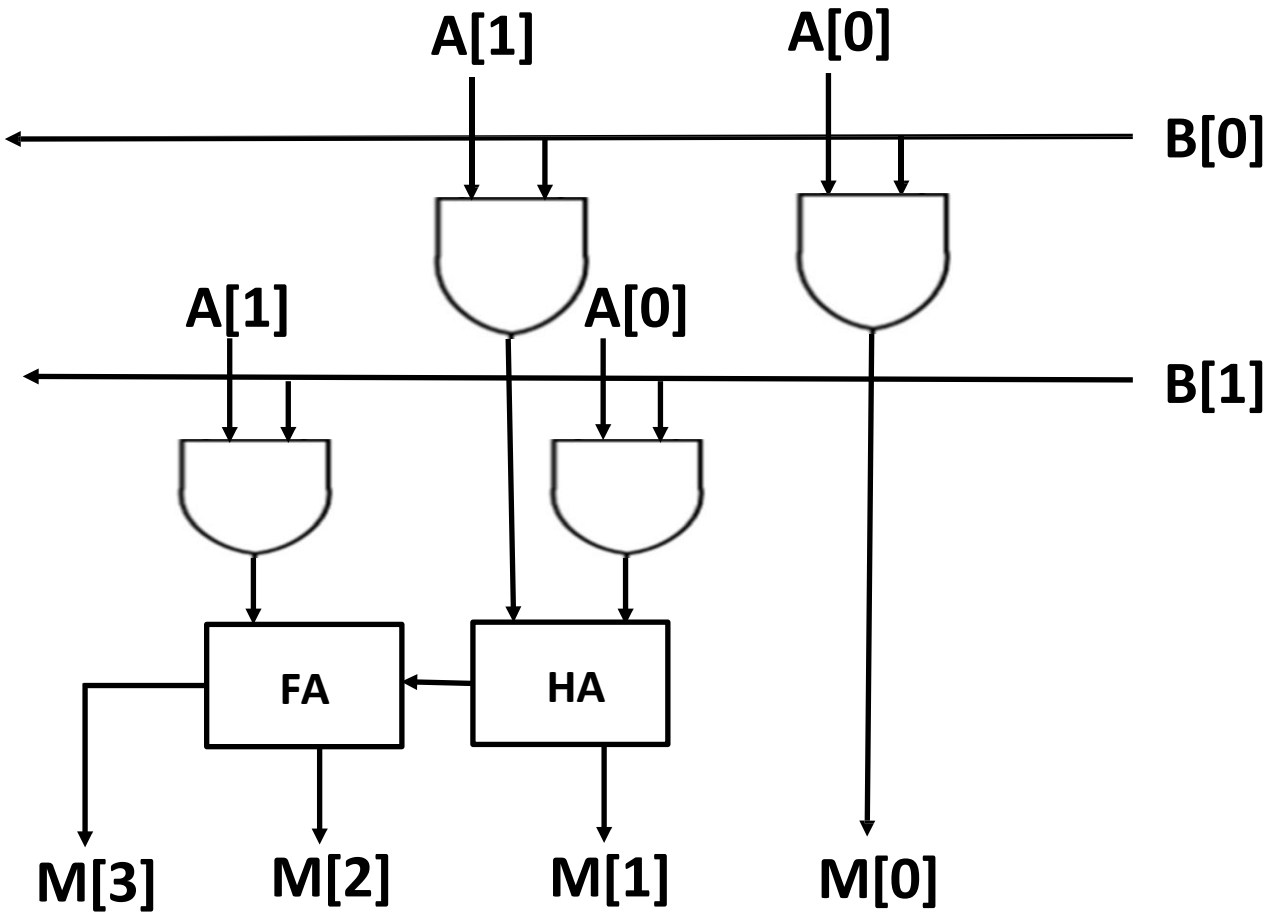


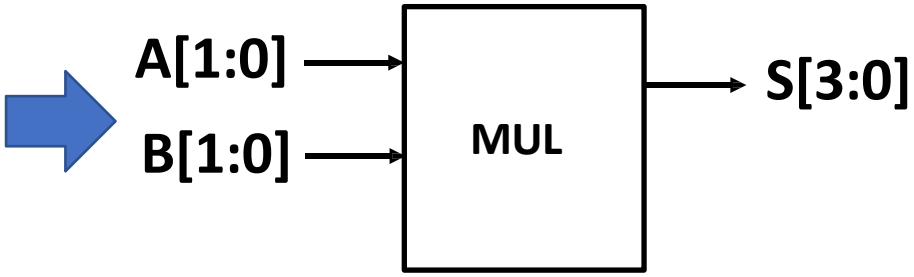
4-bit Unsigned Multiplier

$$
\begin{array}{cccc}
 & & A_1 & A_0 \\
 & & B_1 & B_0 \\
\hline
 & & A_1B_0 & A_0B_0 \\
 & A_1B_1 & A_0B_1 & \\
\hline
M_3 & M_2 & M_1 & M_0
\end{array}
$$

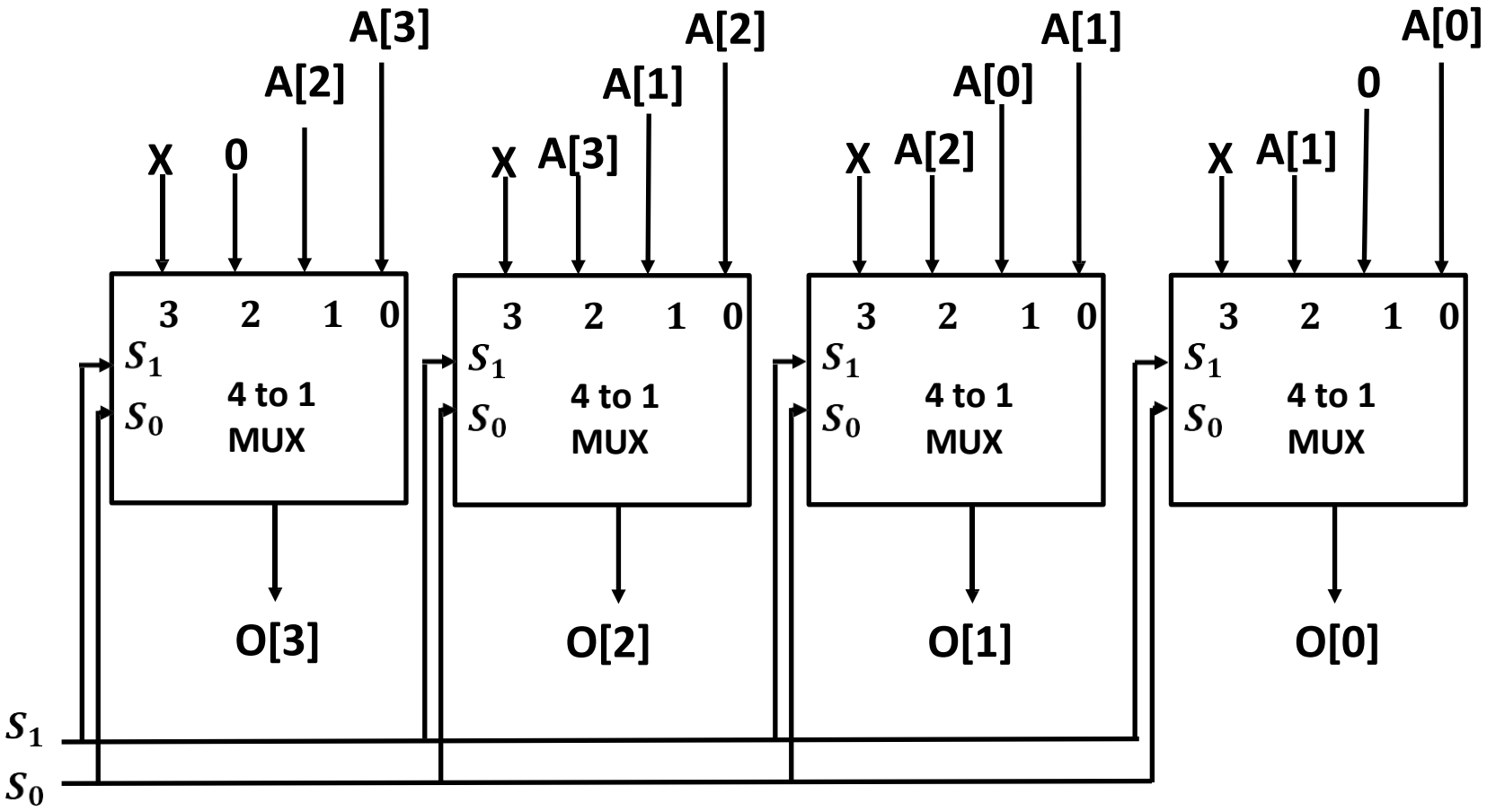


4-bit Unsigned Multiplier





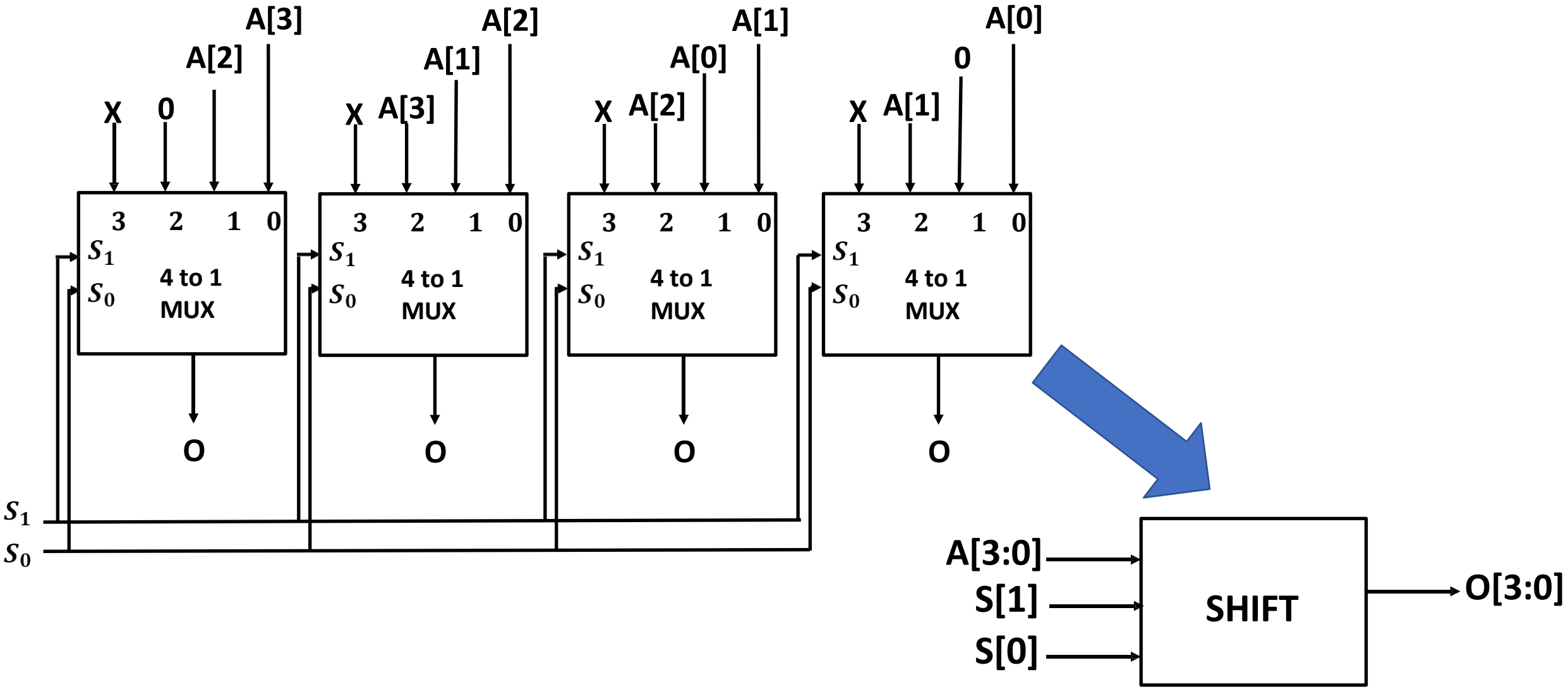
4-bit Shifter



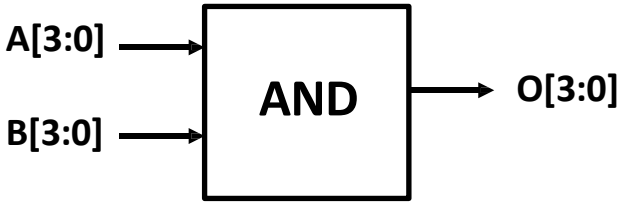
S_1	S_2	Output	Operation
0	0	**A[3]A[2]A[1]A[0]**	No Shift
0	1	**A[2]A[1]A[0] 0**	Left Shift
1	0	**0 A[3]A[2]A[1]**	Right Shift
1	1	X	X

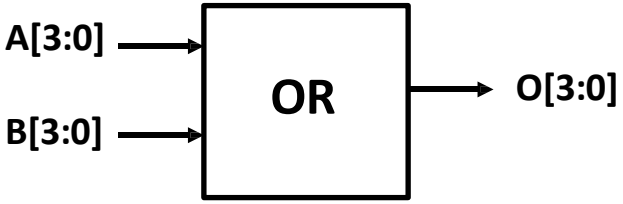
Input: A[3]A[2]A[1]A[0]
Right shift: 0 A[3]A[2]A[1]
Left shift: A[2]A[1]A[0] 0

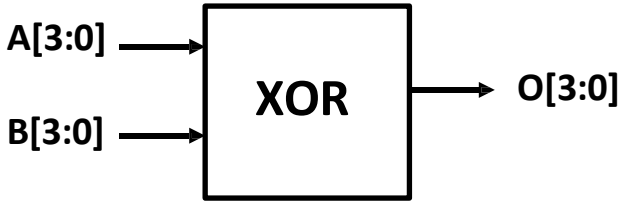
4-bit Shifter

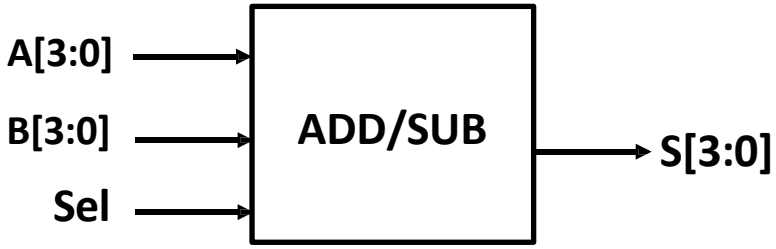


All the circuits so far

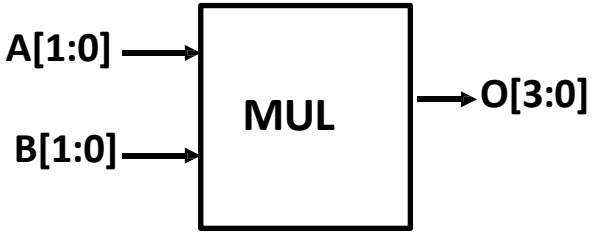


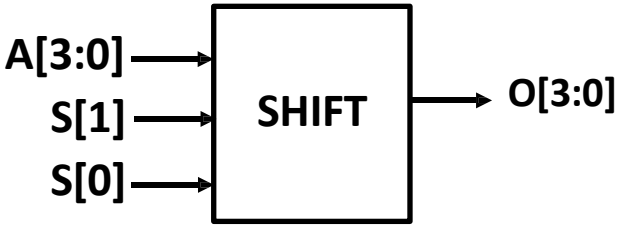






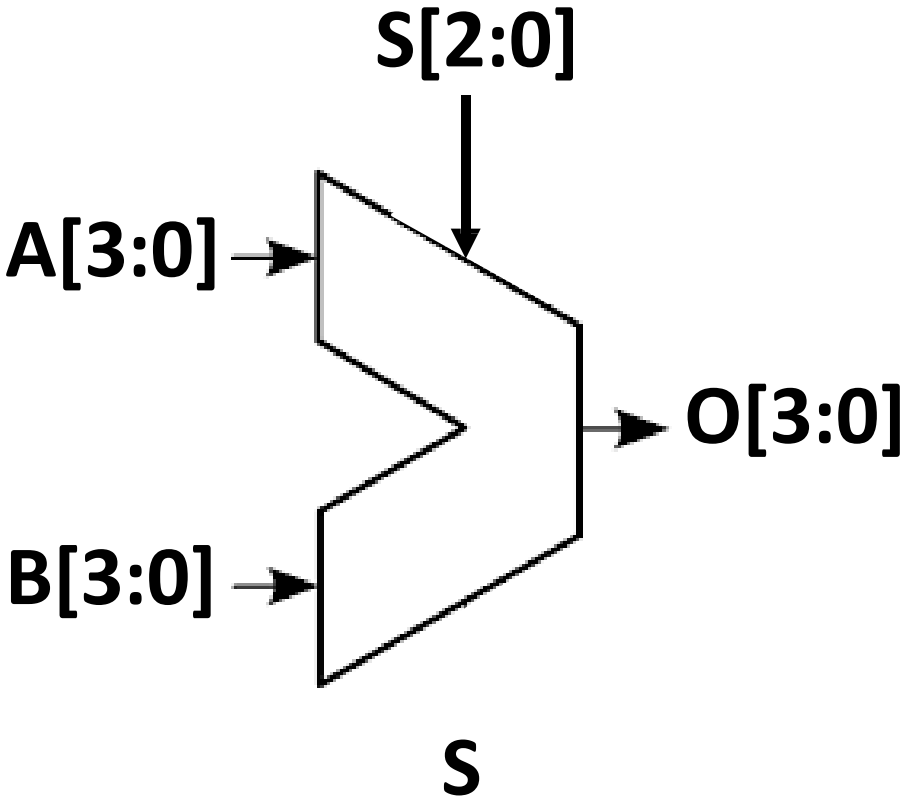
If $\mathbf{sel} = \mathbf{0},\ \mathbf{S} = \mathbf{ADD}$
If $\mathbf{sel} = \mathbf{1},\ \mathbf{S} = \mathbf{SUB}$





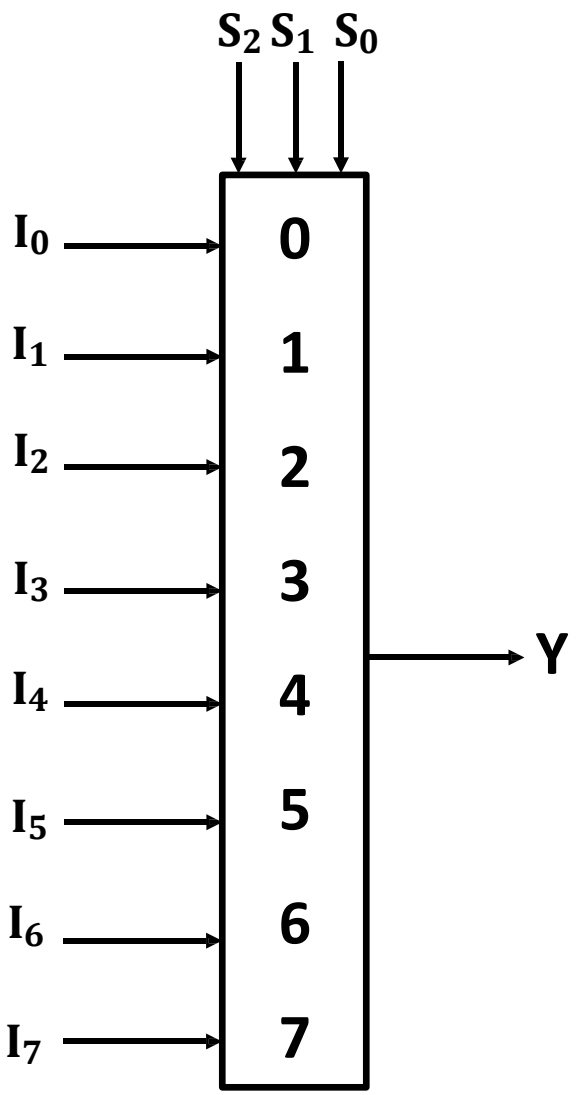
If $\mathbf{S_1} = \mathbf{0}, \mathbf{S_0} = \mathbf{1}, \mathbf{S} = \mathbf{LEFT\ SHIFT}$
If $\mathbf{S_1} = \mathbf{1}, \mathbf{S_0} = \mathbf{0}, \mathbf{S} = \mathbf{RIGHT\ SHIFT}$

4-bit ALU Circuit



4-bit ALU Circuit

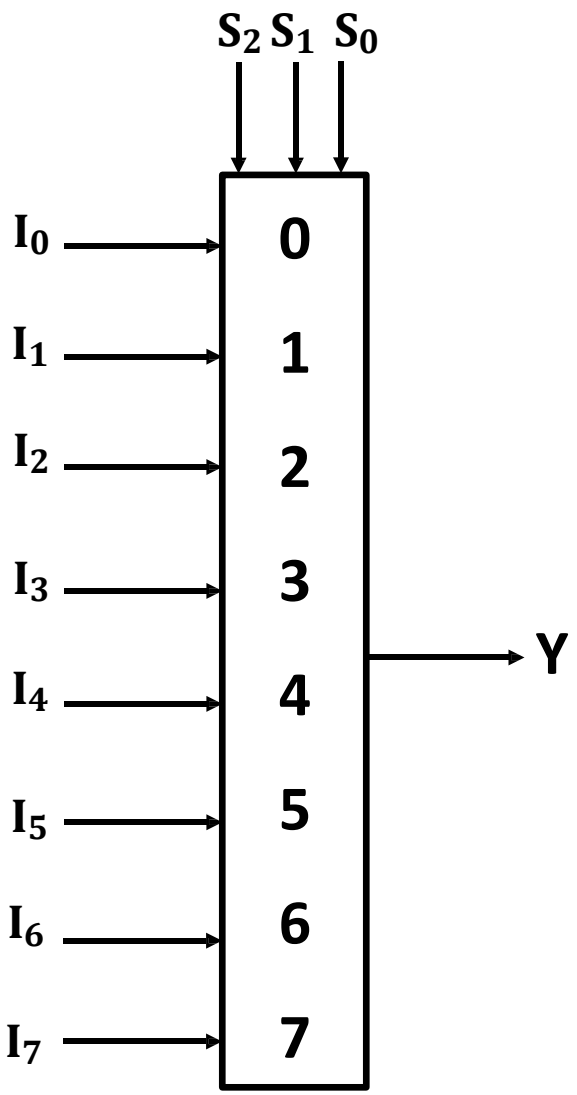
8 to 1 MUX



But this MUX can handle only 1 bit.
How can we build 4-bit MUX?

4-bit ALU Circuit

8 to 1 MUX

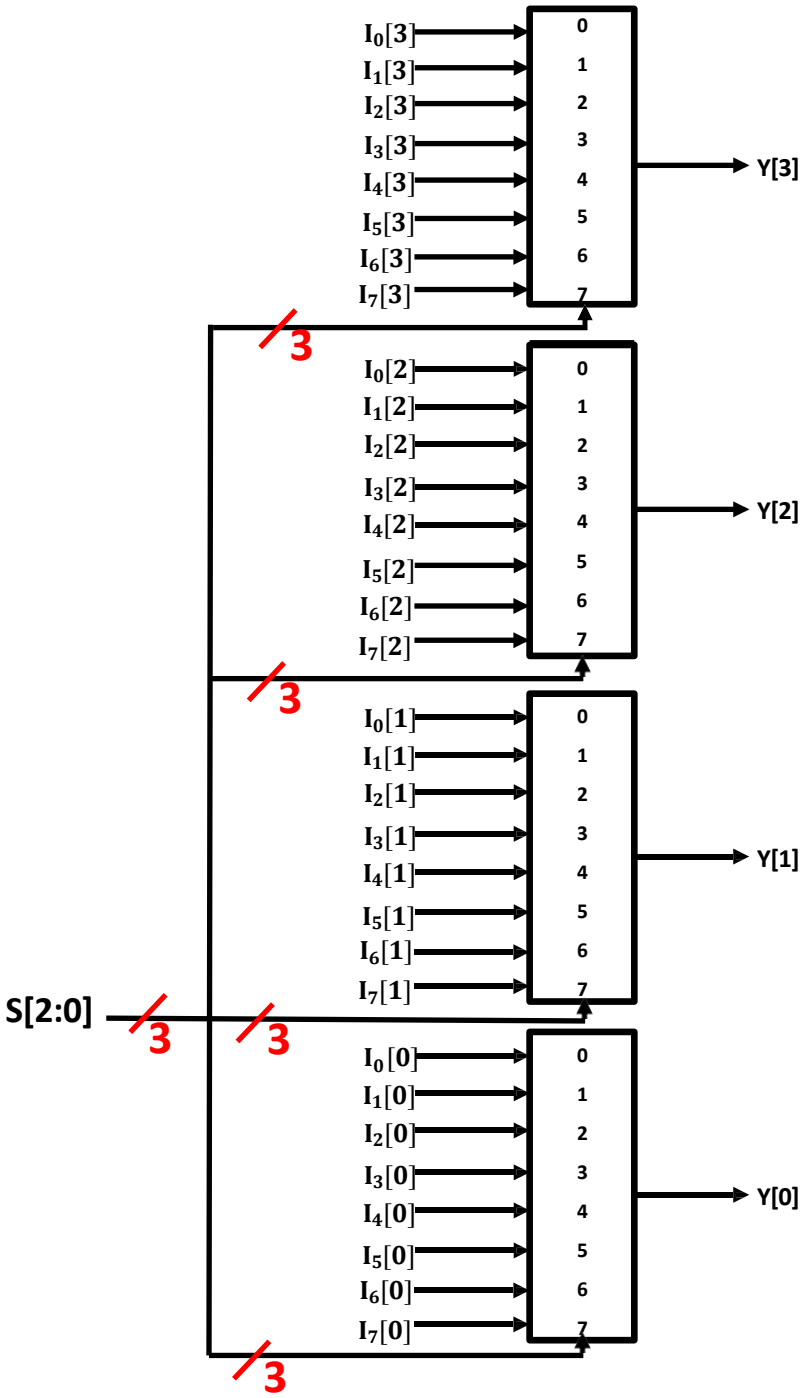


But this MUX can handle only 1 bit. How can we build 4-bit MUX?

Solution is to use FOUR 8 to 1 MUX for 4 input lines and 4 output lines.

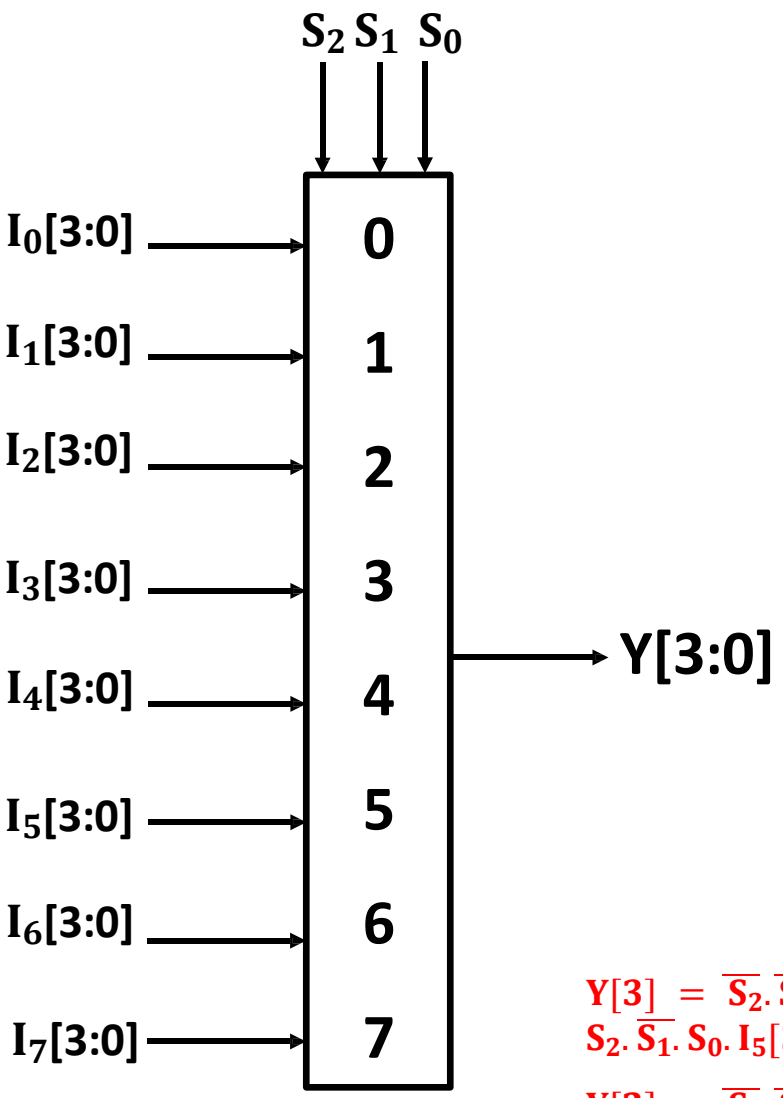
4-bit ALU Circuit

4-bit 8 to 1 MUX



4-bit ALU Circuit

8 to 1 MUX

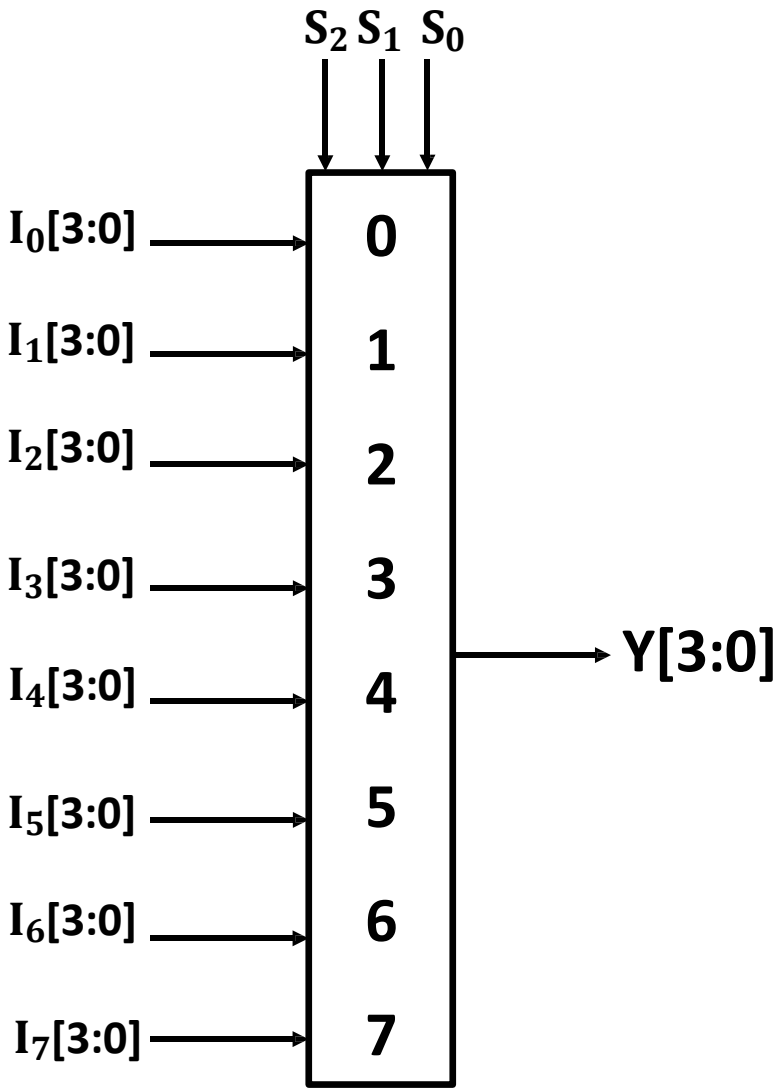


S_2	S_1	S_0	$Y[3]$	$Y[2]$	$Y[1]$	$Y[0]$
0	0	0	$I_0[3]$	$I_0[2]$	$I_0[1]$	$I_0[0]$
0	0	1	$I_1[3]$	$I_1[2]$	$I_1[1]$	$I_1[0]$
0	1	0	$I_2[3]$	$I_2[2]$	$I_2[1]$	$I_2[0]$
0	1	1	$I_3[3]$	$I_3[2]$	$I_3[1]$	$I_3[0]$
1	0	0	$I_4[3]$	$I_4[2]$	$I_4[1]$	$I_4[0]$
1	0	1	$I_5[3]$	$I_5[2]$	$I_5[1]$	$I_5[0]$
1	1	0	$I_6[3]$	$I_6[2]$	$I_6[1]$	$I_6[0]$
1	1	1	$I_7[3]$	$I_7[2]$	$I_7[1]$	$I_7[0]$

$$Y[3] = \overline{S_2}.\overline{S_1}.\overline{S_0}.I_0[3] + \overline{S_2}.\overline{S_1}.S_0.I_1[3] + \overline{S_2}.S_1.\overline{S_0}.I_2[3] + \overline{S_2}.S_1.S_0.I_3[3] + S_2.\overline{S_1}.\overline{S_0}.I_4[3] + S_2.\overline{S_1}.S_0.I_5[3] + S_2.S_1.\overline{S_0}.I_6[3] + S_2.S_1.S_0.I_7[3]$$

$$Y[2] = \overline{S_2}.\overline{S_1}.\overline{S_0}.I_0[2] + \overline{S_2}.\overline{S_1}.S_0.I_1[2] + \overline{S_2}.S_1.\overline{S_0}.I_2[2] + \overline{S_2}.S_1.S_0.I_3[2] + S_2.\overline{S_1}.\overline{S_0}.I_4[2] + S_2.\overline{S_1}.S_0.I_5[2] + S_2.S_1.\overline{S_0}.I_6[2] + S_2.S_1.S_0.I_7[2]$$

4-bit ALU Circuit

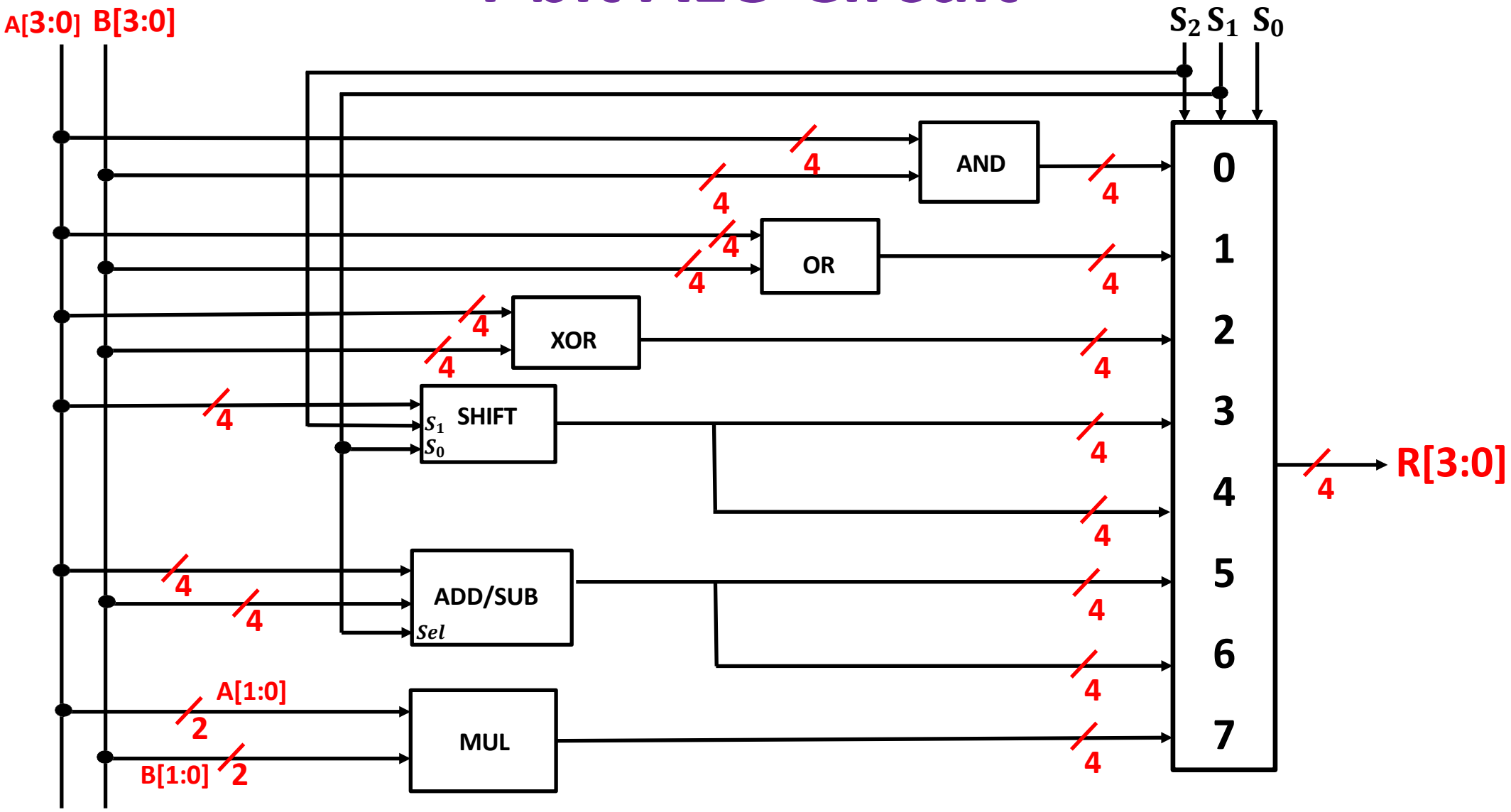


Operation	S_2	S_1	S_0	Y[3]	Y[2]	Y[1]	Y[0]
AND	0	0	0	$I_0[3]$	$I_0[2]$	$I_0[1]$	$I_0[0]$
OR	0	0	1	$I_1[3]$	$I_1[2]$	$I_1[1]$	$I_1[0]$
XOR	0	1	0	$I_2[3]$	$I_2[2]$	$I_2[1]$	$I_2[0]$
LEFT SHIFT	0	1	1	$I_3[3]$	$I_3[2]$	$I_3[1]$	$I_3[0]$
RIGHT SHIFT	1	0	0	$I_4[3]$	$I_4[2]$	$I_4[1]$	$I_4[0]$
ADD	1	0	1	$I_5[3]$	$I_5[2]$	$I_5[1]$	$I_5[0]$
SUB	1	1	0	$I_6[3]$	$I_6[2]$	$I_6[1]$	$I_6[0]$
MUL	1	1	1	$I_7[3]$	$I_7[2]$	$I_7[1]$	$I_7[0]$

If $\mathbf{sel = 0,\ S = ADD}$
If $\mathbf{sel = 1,\ S = SUB}$

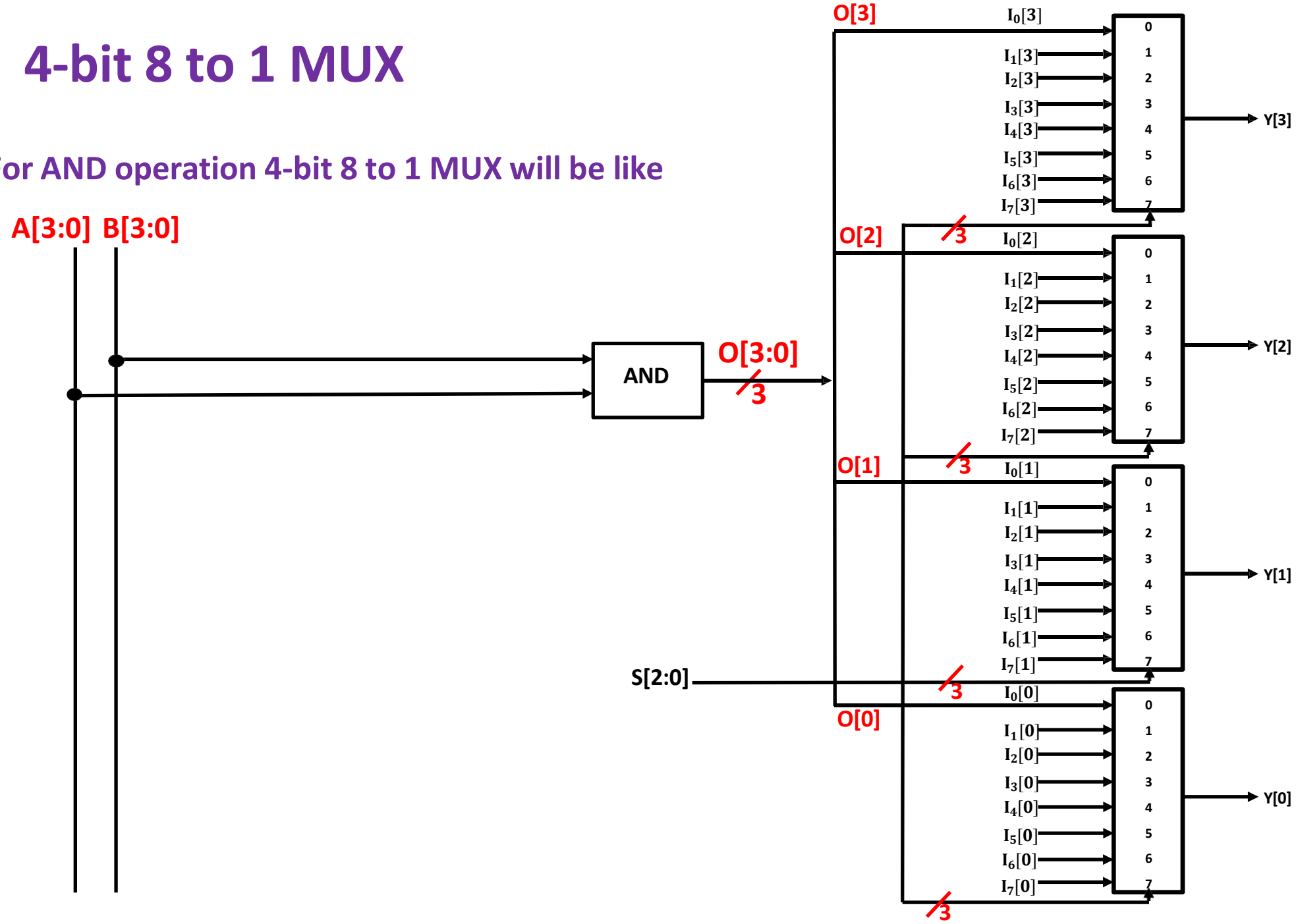
If $\mathbf{S_1 = 0, S_0 = 1, S = LEFT\ SHIFT}$
If $\mathbf{S_1 = 1, S_0 = 0, S = RIGHT\ SHIFT}$

4-bit ALU Circuit



4-bit 8 to 1 MUX

For AND operation 4-bit 8 to 1 MUX will be like



Home Work:

Design a 2-bit ALU that supports following operations: AND, LEFT SHIFT, MUL & SUB.

Calculating FLAG Values in ALU

Question: How can we implement conditional branch instructions like JE, JNE, JG, JL, JLE etc.?

Answer:
We can implement conditional instructions by using FLAG values. Flag values are stored in FLAG Register.

FLAG Register

FLAG register always save state of previous instruction. Its flags will be on/off depending on result of previous instruction.

We learned about 8 flags in 8086 processor. They are:

Sign Flag
Zero Flag
Auxiliary Cary Flag
Parity Flag
Carry Flag
Overflow Flag
Directional Flag
Interrupt Flag
Trap Flag

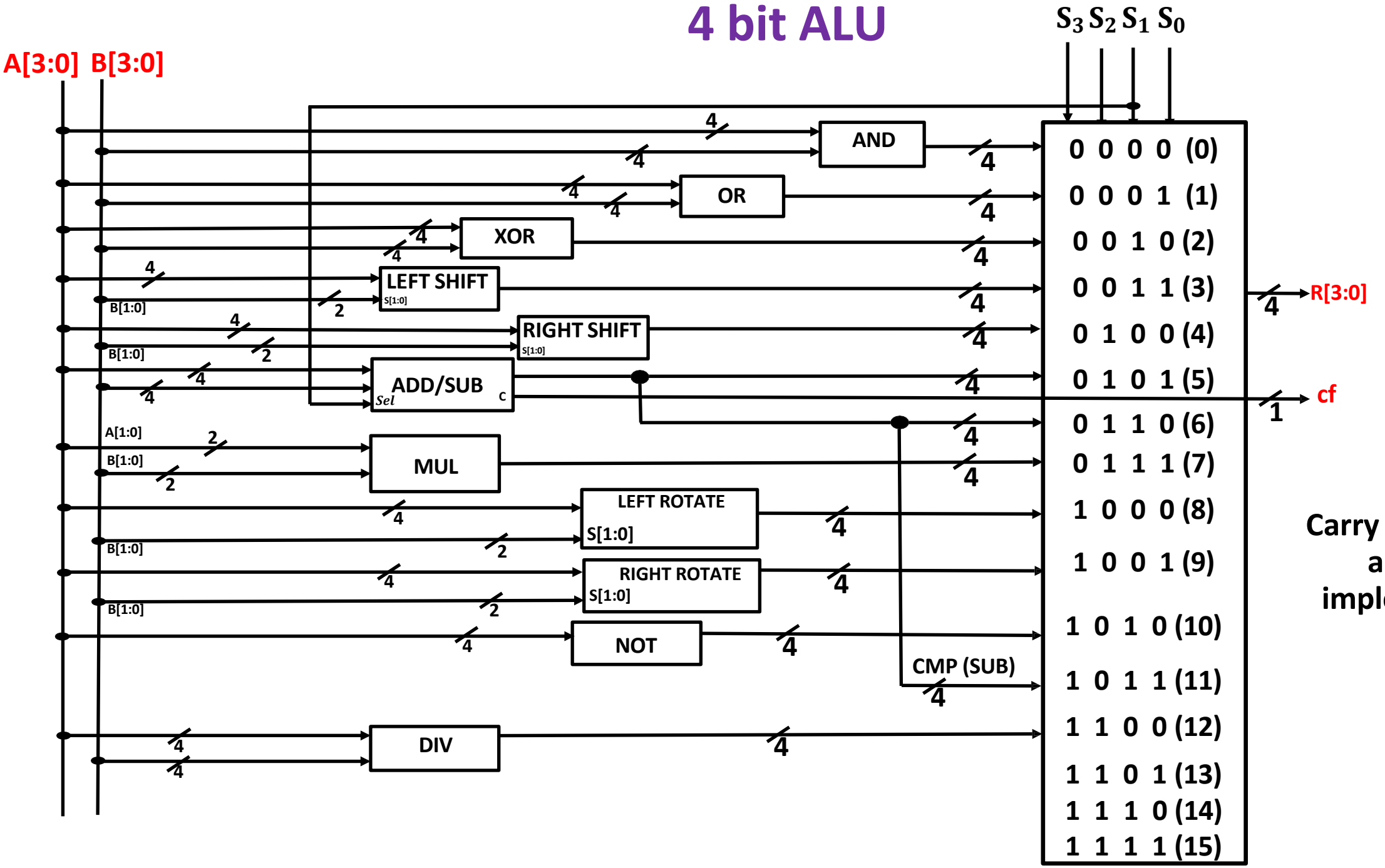
FLAG Register

We will implement 3 flags in our CPU. They are:

1. **Carry Flag (CF):** It will be ON/1 when result of ADD/SUB have carry.
2. **Sign Flag (SF):** It will determine whether result is positive or negative.
3. **Zero Flag (ZF):** It will determine whether result is 0.

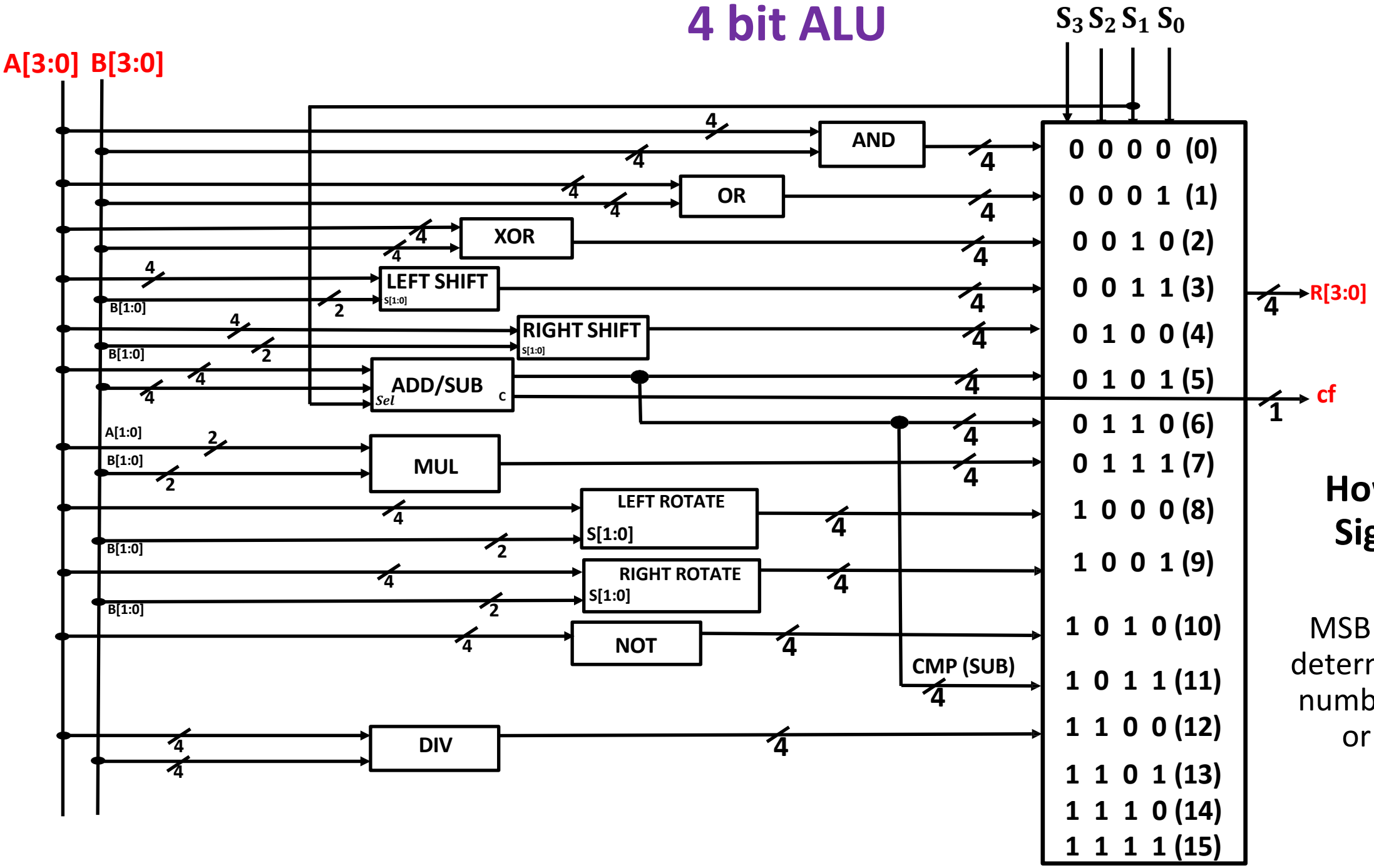
Flag value depends on Result of ALU.

4 bit ALU



Carry Flag (CF) is already implemented.

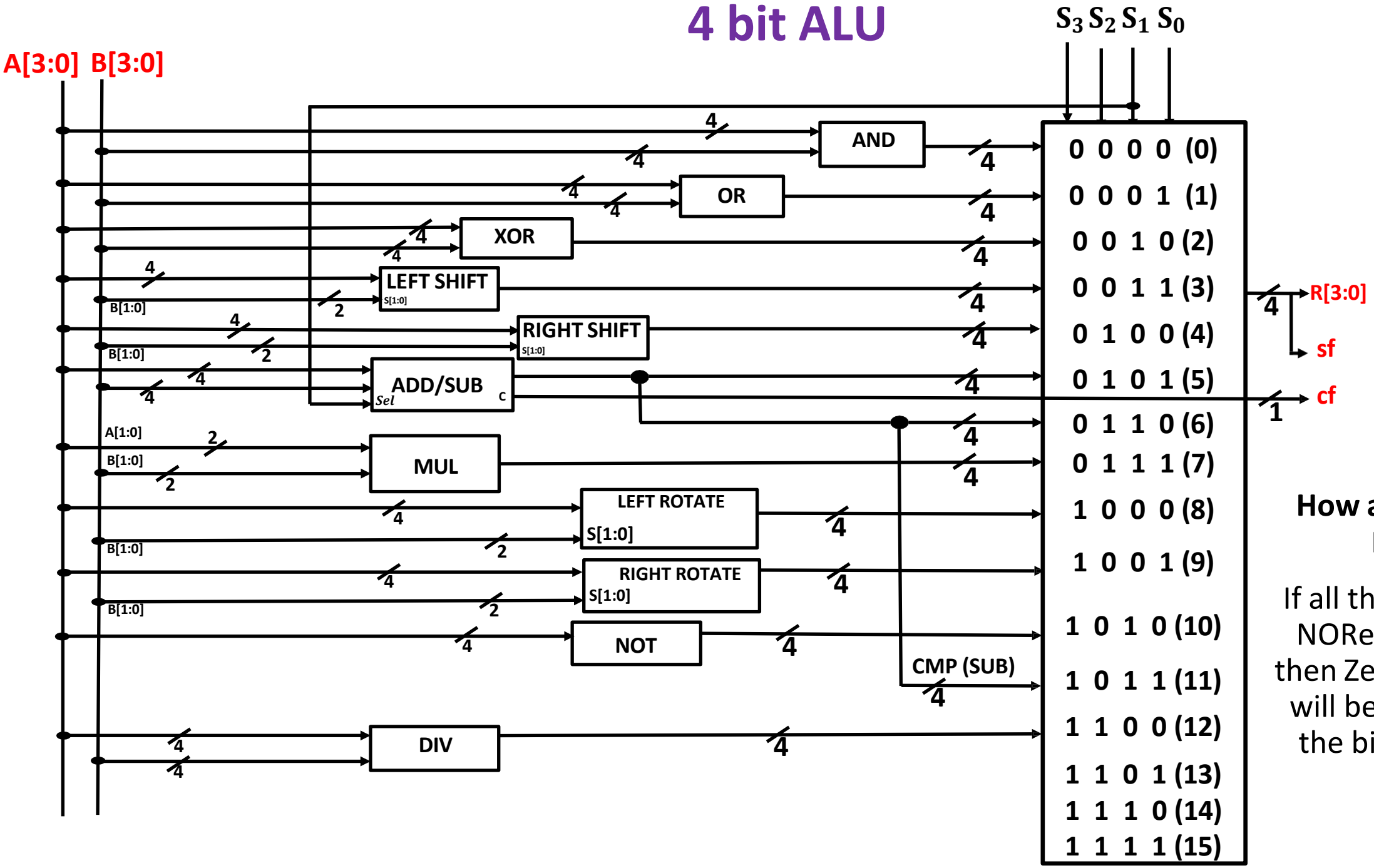
4 bit ALU



How about Sign Flag?

MSB bit of result determine whether number is positive or negative.

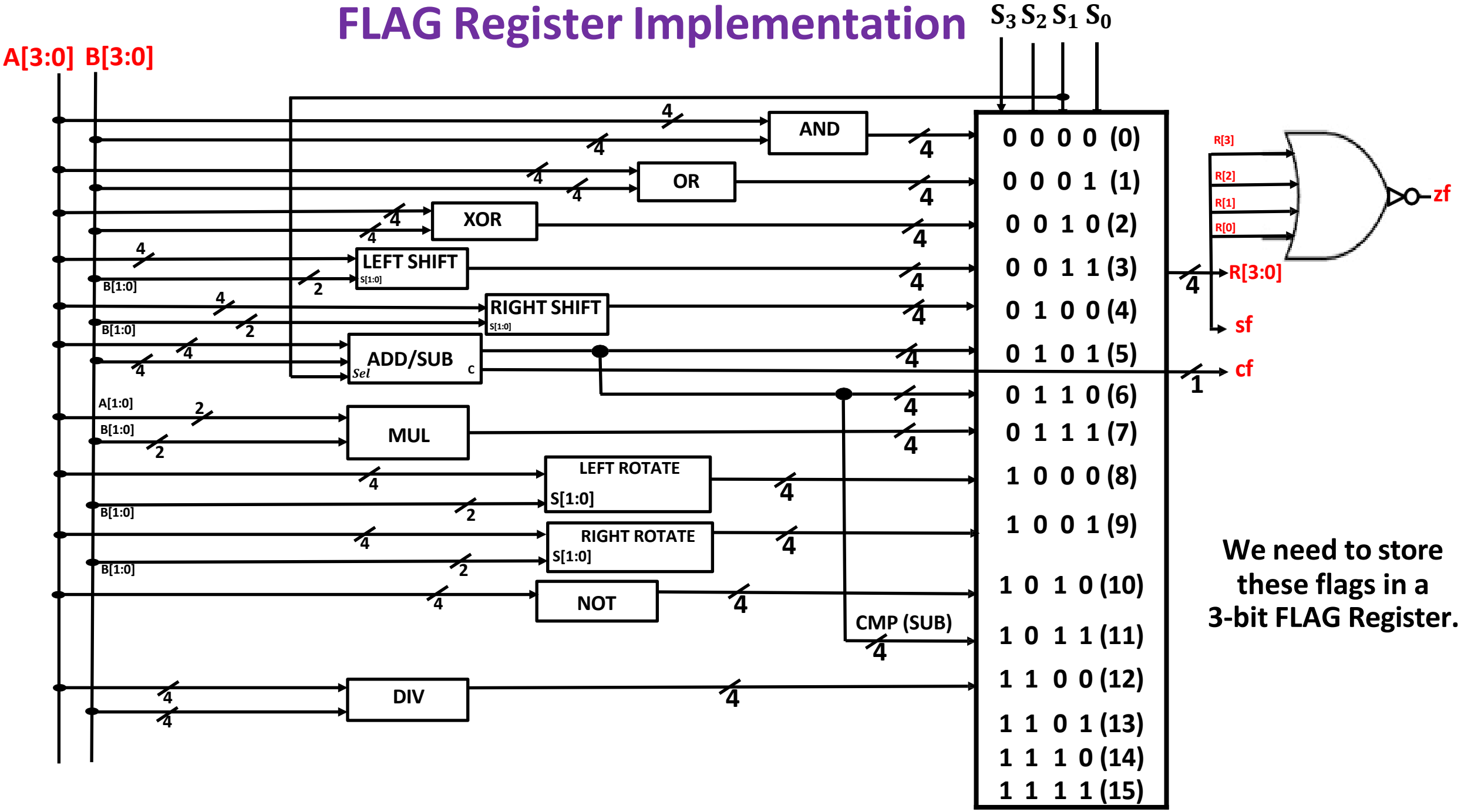
4 bit ALU



How about Zero Flag?

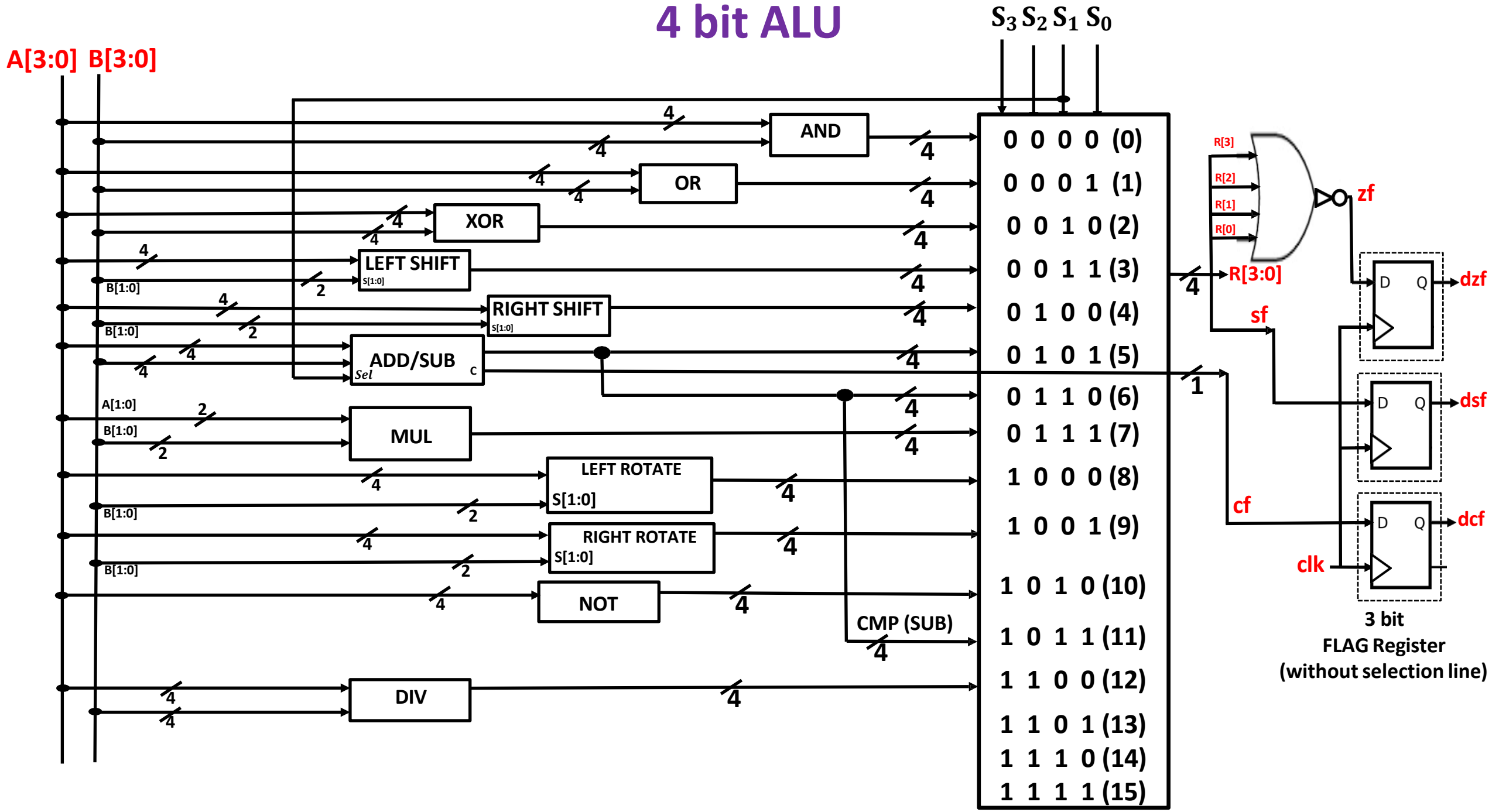
If all the bits of R is NORed together, then Zero Flag value will be 1 only if all the bits of R is 0.

FLAG Register Implementation



We need to store these flags in a 3-bit FLAG Register.

4 bit ALU


A[3:0] B[3:0]
S_3 S_2 S_1 S_0
AND
OR
XOR
LEFT SHIFT
S[1:0]
RIGHT SHIFT
S[1:0]
ADD/SUB
Sel
c
MUL
LEFT ROTATE
S[1:0]
RIGHT ROTATE
S[1:0]
NOT
DIV
CMP (SUB)
B[1:0]
A[1:0]
0 0 0 0 (0)
0 0 0 1 (1)
0 0 1 0 (2)
0 0 1 1 (3)
0 1 0 0 (4)
0 1 0 1 (5)
0 1 1 0 (6)
0 1 1 1 (7)
1 0 0 0 (8)
1 0 0 1 (9)
1 0 1 0 (10)
1 0 1 1 (11)
1 1 0 0 (12)
1 1 0 1 (13)
1 1 1 0 (14)
1 1 1 1 (15)
R[3]
R[2]
R[1]
R[0]
R[3:0]
zf
sf
cf
clk
D Q
dzf
dsf
dcf
3 bit
FLAG Register
(without selection line)

Example: ALU

Question: Design a 4-bit ALU that supports RIGHT ROTATE and DIV (Unsigned) operations.

Answer:

4-bit RIGHT ROTATE Circuit:

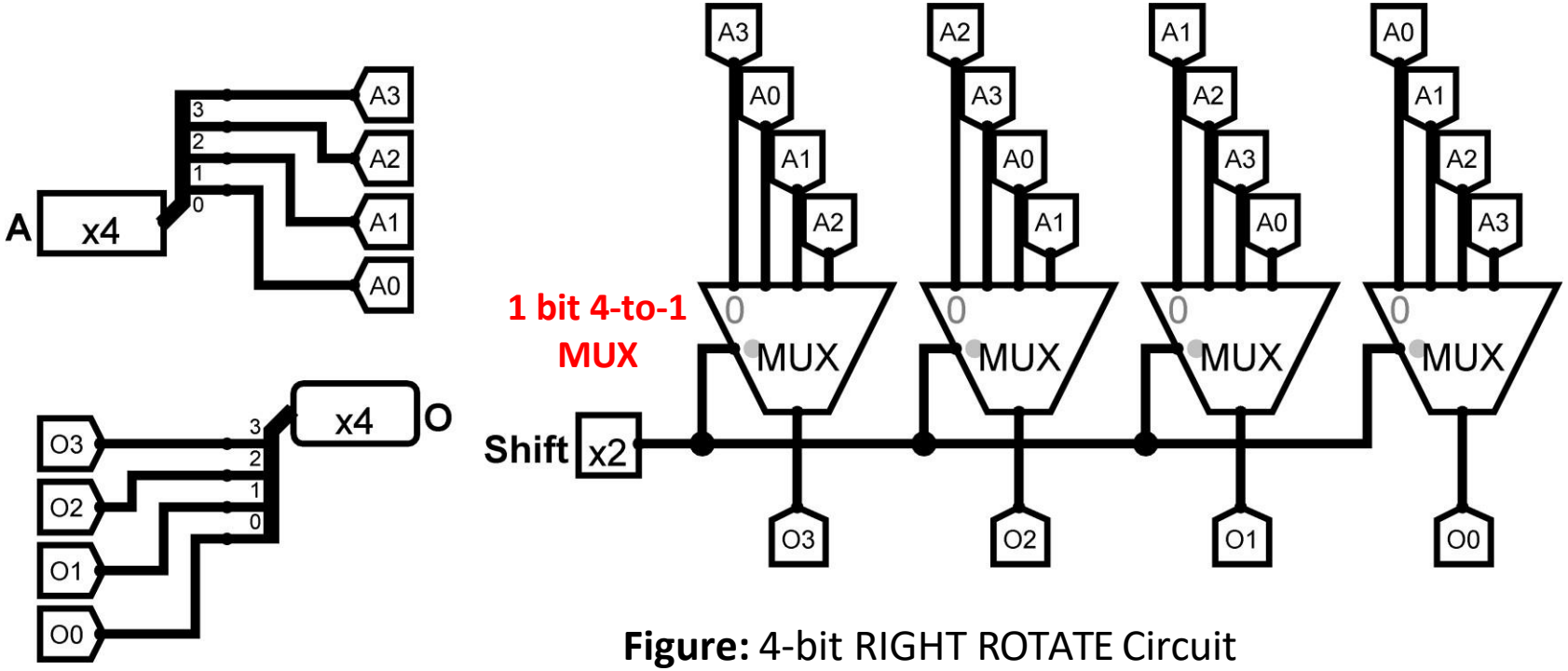


Figure: 4-bit RIGHT ROTATE Circuit

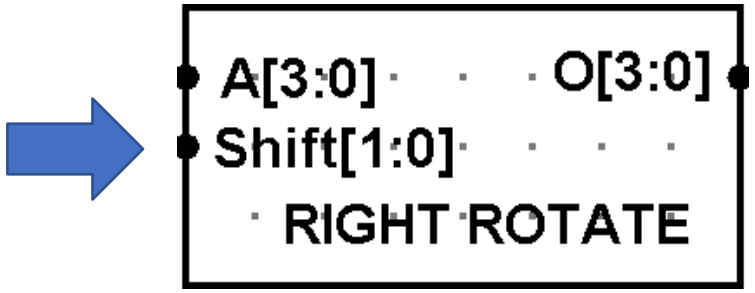


Figure: 4-bit RIGHT ROTATE Chip

Example: ALU

4-bit DIV (Unsigned) Circuit:

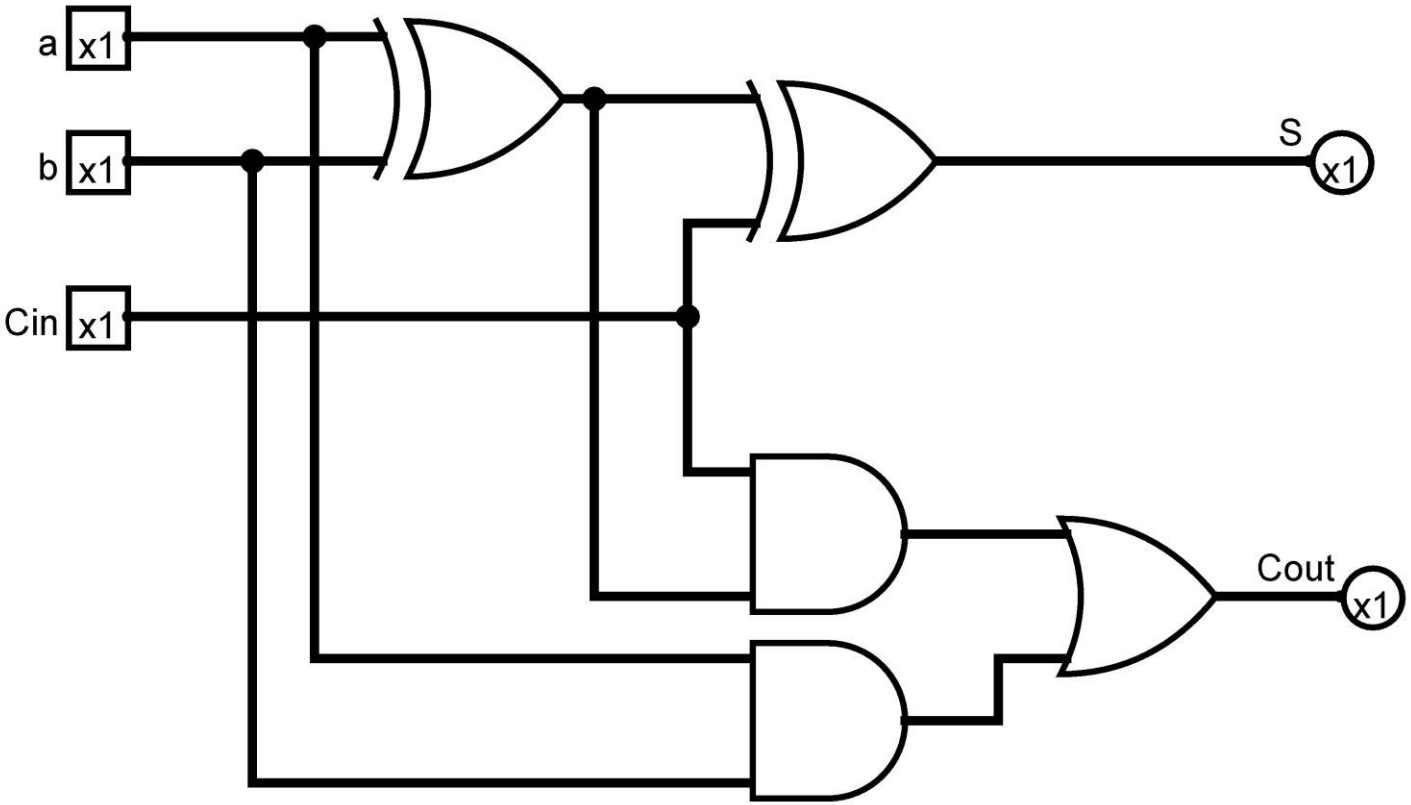


Figure: 1-bit Full Adder Circuit

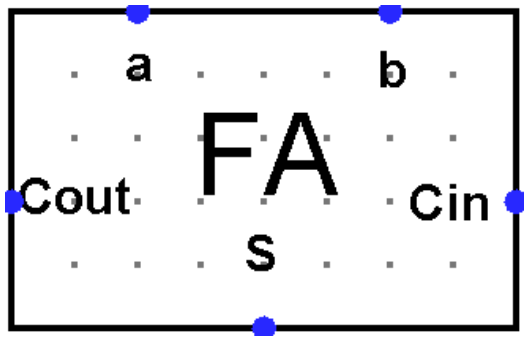


Figure: 1-bit Full Adder Chip

Example: ALU

4-bit DIV (Unsigned) Circuit:

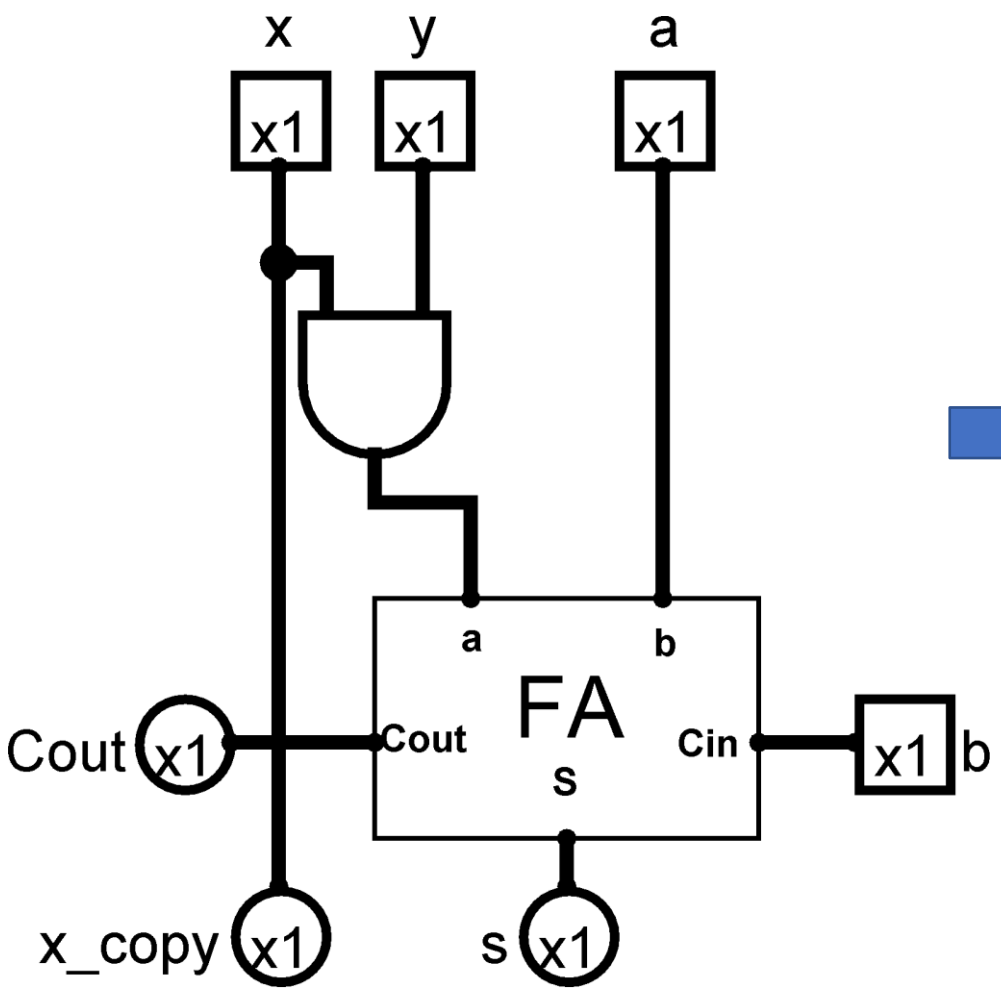


Figure: Cell M (Divider Block)

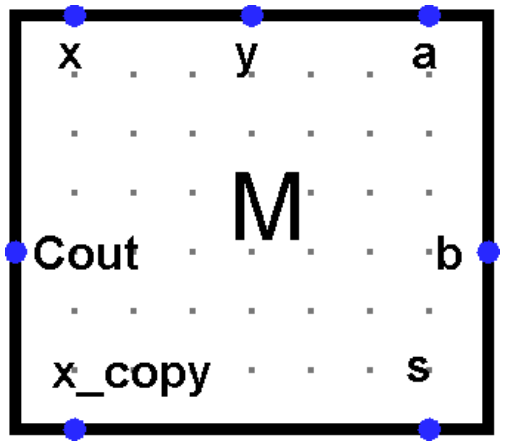


Figure: Cell M (Divider Block) Chip

Example: ALU

4-bit DIV (Unsigned) Circuit:

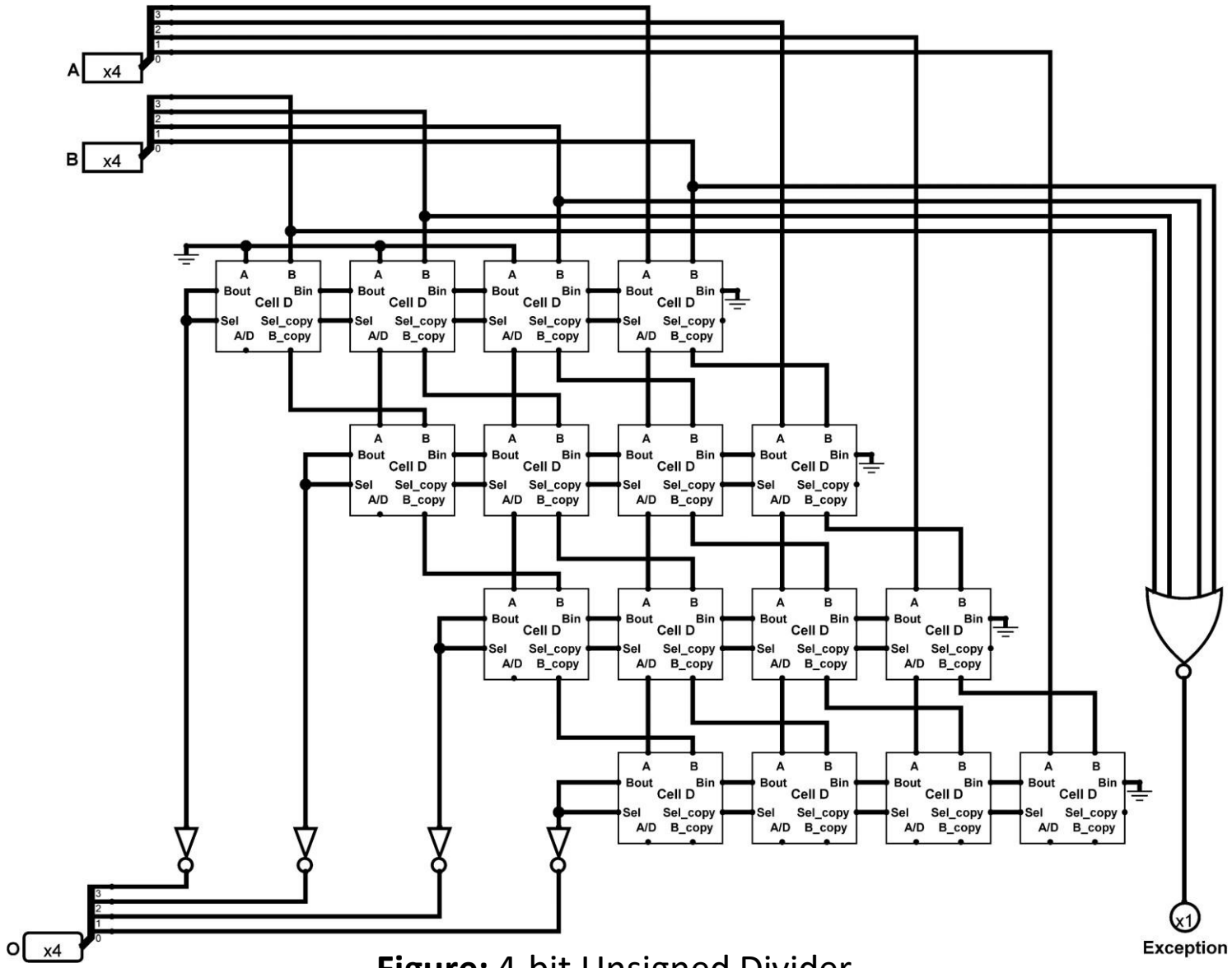


Figure: 4-bit Unsigned Divider

Example: ALU

4-bit DIV (Unsigned) Circuit:

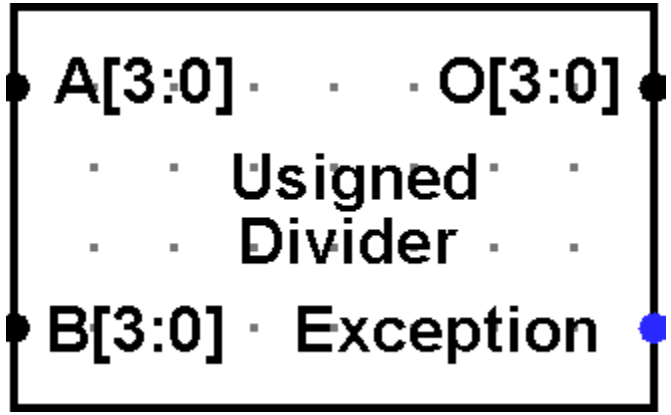


Figure: 4-bit Unsigned Divider Chip

Example: ALU

4-bit ALU Circuit:

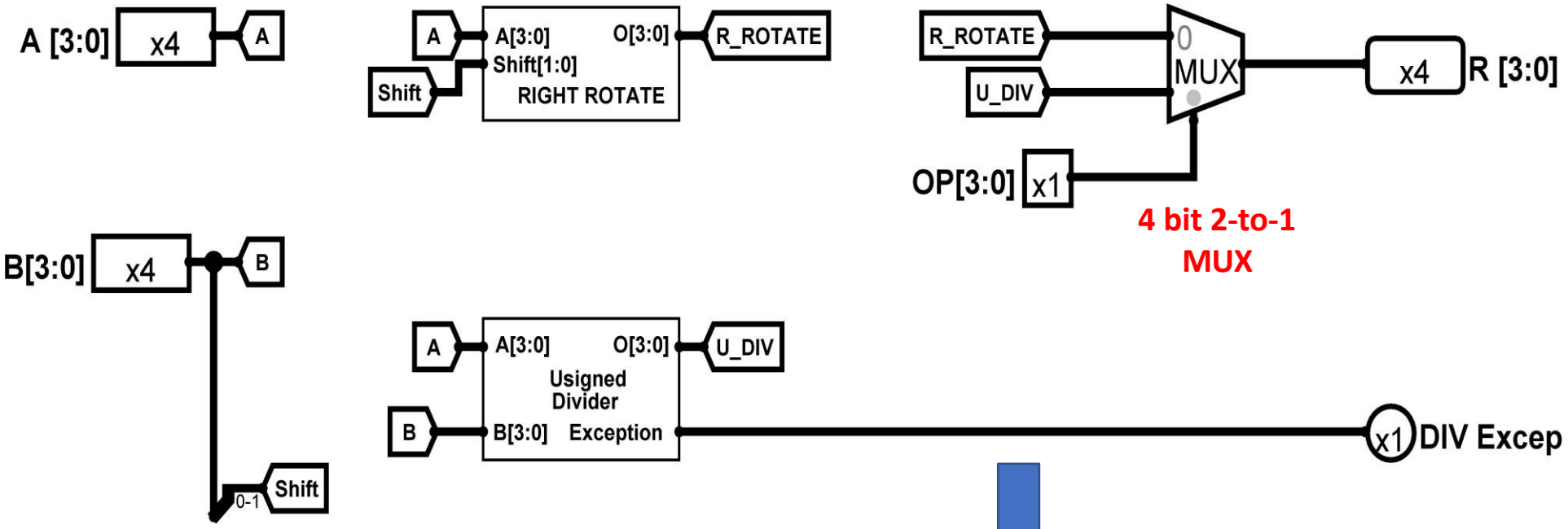


Figure: 4-bit ALU Circuit that supports RIGHT ROTATE and DIV

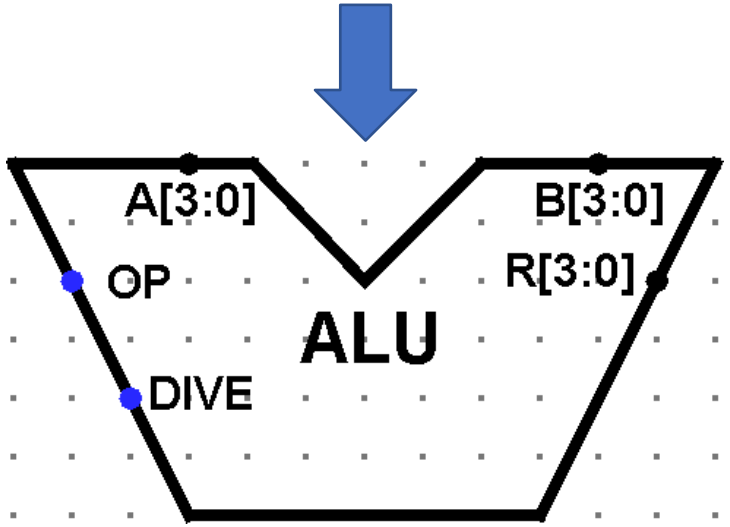


Figure: 4-bit ALU Circuit Chip

Exercises

1. Draw/Design/Implement an 1-bit/2-bit/3-bit/4-bit/5-bit ALU that supports following operations:

   i. ADD /
   ii. SUB /
   iii. MUL /
   iv. DIV /
   v. LEFT SHIFT /
   vi. RIGHT SHIFT /
   vii. LEFT ROTATE /
   viii. RIGHT ROTATE /
   ix. AND /
   x. OR /
   xi. XOR /
   xii. NOT /
   xiii. CMP

Thank You ☺